

Contents

1. Setup and Utilities	3
Template	3
Precompile stdc++.h	3
Pragmas	3
Random mt19937	3
Direction Arrays	3
Custom Comparator	4
Fast HashMap (gp_hash_table)	4
__int128_t	4
2. Data Structures	5
2D Prefix Sum	5
BIT, Fenwick Tree	5
BIT Range / Fenwick Range	5
2D BIT / 2D fenwick	6
Segment Tree	7
Segment Tree (Recursive)	8
2D Segment Tree	8
Lazy Segment Tree	10
Dynamic Lazy Segment Tree	11
Segment Tree Beats	12
Dynamic Persistent Segment Tree	16
Wavelet Tree	16
Implicit Treap	17
Sparse Table	19
2D Sparse Table	20
DSU	21
Persistent DSU	22
Bipartite DSU	23
Monotonic Stack / Queue	23
Ordered Data Structures (pb_ds)	24
BucketList	24
Full Dynamic Array	25
Count below	26
3. Graphs and Trees	28
Bellman Ford	28
Floyd Tricks	28
Topological Sort	29
Bipartite Check	30
Tree Diameter	30
Tree	30
Binary Lifting	32
LCA	32
DSU on Trees	33
SCC / Strongly Connected Componenets	34
WLCA	35
Rerooter	36
Dynamic Connectivity	38
Max Flow (Dinik)	40
Max Bipartite Matching (Karp) [with building]	41
4. Math and Number Theory	43
Math	43
ModInt	47
Sieve / PHI up to n / 2D gcd()	47
Sieve up to 1e9	49

Egcd, Linear Diaphontine	50
CRT	51
MillerRabin	51
Number of Divisors up to $1e18$	52
$n \bmod 1 + n \bmod 2 + n \bmod 3 + \dots + n \bmod m$	54
n-th Fib Number	54
Long Division	54
Floor Values	54
Combinatorics	54
nCr, nPr without precomputation	55
NCR table	55
Catalan numbers	56
Matrix Exponentiation	57
K-th permutation	58
Permutation Index	58
Berlekamp Massey	58
5. Strings	61
Rolling Hash	61
Tree Hash (rooted/unrooted)	62
Z-Algo	63
Largest Lexicographical Substring	63
Manacher	63
Aho Corasick Algorithm	64
XOR Trie	65
Suffix Array	67
Suffix Automaton	68
Suffix Automaton Subproblems	71
6. Dynamic Programming	72
LIS	72
0/1 Knapsack	72
Edit Distance	73
Longest Common Subsequence	73
Shortest Common Supersequence	74
Count Subsets Sum to K	74
Hamiltonian Paths	75
Kadane	75
Dynamic max subarray sum	75
Meet in the middle	76
Digit DP	76
7. Bit Manipulation	78
Xor Basis	78
N-SAT	79
Max Xor Subset In Range	81
and (&) in range $[l, r]$	82
Bit Twiddle Permute	83
8. Game Theory and Sequences	84
Mex Calculator	84
Remove Game	84
K-th Balanced Bracket Sequence	84
Next Balanced Sequence	85
9. Geometry	86
Geometry Notes	86
Geometry (X, Y, dot, cross)	87

1. Setup and Utilities

Template

```
1 #include "bits/stdc++.h"
2 using namespace std;
3 using ll = long long;
4 #define int ll
5 #define endl '\n'
6 void solve() {
7 }
8 signed main() {
9     cin.tie(0)->sync_with_stdio(0);
10    cin.exceptions(cin.failbit);
11    int tt = 1;
12    // cin >> tt;
13    while (tt--) {
14        solve();
15        cout << '\n';
16    }
17    return 0;
18 }
```

Precompile stdc++.h

```
1 sudo g++ -x c++-header -std=c++17 -O0 stdc++.h -o stdc++.h.gch
```

Pragmas

```
1 #pragma GCC optimize("Ofast")
2 #pragma GCC optimize ("unroll-loops")
3 #pragma GCC target("sse,sse2,sse3,ssse3,sse4,popcnt,abm,mmx,avx,tune=native")
```

Random mt19937

```
1 mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
2 ll rnd(ll l, ll r) {
3     static mt19937_64 gen(chrono::steady_clock::now().time_since_epoch().count());
4     return uniform_int_distribution<ll>(l, r)(gen);
5 }
```

Direction Arrays

```
1 int dx[8] = { 2, 1, -1, -2, -2, -1, 1, 2 };
2 int dy[8] = { 1, 2, 2, 1, -1, -2, -2, -1 }; // knight
3
4 int dx[8] = {-1,0,1,-1,1,-1,0,1};
5 int dy[8] = {-1,-1,-1,0,0,1,1,1}; // king
6
7 int dx[4] = {1, -1, 0, 0};
8 int dy[4] = {0, 0, -1, 1};
9 string direction = "DULR";
```

Custom Comparator

```
1 struct cmp {
2     bool operator() (int a, int b) const {
3         return ...;
4     }
5 };
6 set<int, cmp> s;
```

Fast HashMap (gp_hash_table)

```
1 #include <ext/pb_ds/assoc_container.hpp>
2 using namespace __gnu_pbds;
3 struct chash {
4     const int RANDOM = (ll)(make_unique<char>().get()) ^
5         chrono::high_resolution_clock::now().time_since_epoch().count();
6
7     static unsigned long long hash_f(unsigned long long x) {
8         x += 0x9e3779b97f4a7c15;
9         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
10        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
11        return x ^ (x >> 31);
12    }
13    static unsigned hash_combine(unsigned a, unsigned b) {
14        return a * 31 + b;
15    }
16    int operator()(int x) const {
17        return hash_f(x)^RANDOM;
18    }
19 };
20 gp_hash_table<int, int, chash> table;
```

__int128_t

```
1 istream& operator>>(istream& is, __int128_t& v) {
2     string s; is >> s; v = 0;
3     for (char c : s) if (isdigit(c)) v = v * 10 + c - '0';
4     if (s[0] == '-') v = -v;
5     return is;
6 }
7 ostream& operator<<(ostream& os, __int128_t v) {
8     if (!v) return os << "0";
9     if (v < 0) os << '-', v = -v;
10    string s;
11    while (v) s += '0' + v % 10, v /= 10;
12    reverse(s.begin(), s.end());
13    return os << s;
14 }
```

2. Data Structures

2D Prefix Sum

```
1 // construction
2 for (int i = 1; i <= N; i++) {
3     for (int j = 1; j <= N; j++) {
4         prefix[i][j] =
5             arr[i][j]
6             + prefix[i - 1][j]
7             + prefix[i][j - 1]
8             - prefix[i - 1][j - 1];
9     }
10 }
11
12 // query
13 pfx[to_row][to_col]
14 - pfx[from_row - 1][to_col]
15 - pfx[to_row][from_col - 1]
16 + pfx[from_row - 1][from_col - 1];
17
18 // partial: add value v to subrectangle [from_row, from_col] to [to_row, to_col]
19 diff[from_row][from_col] += v;
20 diff[from_row][to_col + 1] -= v;
21 diff[to_row + 1][from_col] -= v;
22 diff[to_row + 1][to_col + 1] += v;
23 // then construct diff, grid[i][j] += diff[i][j];
```

BIT, Fenwick Tree

```
1 struct BIT { // 0-based
2     int n;
3     vector<ll> tree;
4
5     BIT(int size) : n(size + 2), tree(n + 1) {}
6     void add(int i, ll val) {
7         for(i++; i <= n; i += i & -i) tree[i] += val;
8     }
9     ll query(int i) {
10         ll sum = 0;
11         for (i++; i > 0; i &= i - 1) sum += tree[i];
12         return sum;
13     }
14     int lower_bound(ll target) {
15         int i = 0;
16         ll curr = 0;
17         for (int mask = 1 << __lg(n); mask > 0; mask >>= 1) {
18             if (i + mask <= n && curr + tree[i + mask] < target) {
19                 curr += tree[i += mask];
20             }
21         }
22         return i;
23     }
24 };
```

BIT Range / Fenwick Range

```

1  template<typename T>
2  class BitR { // 0-based
3      int n;
4      vector<T> f, s;
5      void add(vector<T> &a, int i, T val) {
6          for(; i < n; i += i & -i)
7              a[i] += val;
8      }
9  public:
10     BitR(int n) : n(n + 5), f(n + 6), s(n + 6) { }
11
12     void add(int i, T val) { add(s, i + 1, -val); }
13
14     T query_point(int i) {
15         if (!i) return query(0);
16         return query(i) - query(i - 1);
17     }
18     void set(int i, T val) {
19         int c = query_point(i);
20         add(i, val - c);
21     }
22
23     void add(int l, int r, T val) {
24         l++, r++;
25         add(f, l, val);
26         add(f, r + 1, -val);
27         add(s, l, val * (l - 1));
28         add(s, r + 1, -val * r);
29     }
30
31     T query(int ii) {
32         ii++;
33         T sum = 0;
34         int i = ii;
35         for(; i > 0; i ^= i & -i)
36             sum += f[i];
37         sum *= ii;
38         i = ii;
39         for(; i > 0; i ^= i & -i)
40             sum -= s[i];
41         return sum;
42     }
43
44     T range(int l, int r) { return query(r) - query(l - 1); }
45 };

```

2D BIT / 2D fenwick

```

1  template<typename T>
2  struct BIT2D { // 1-based
3      int n, m;
4      vector<vector<T>> tree;
5
6      BIT2D(int n, int m) : n(n), m(m), tree(n + 2, vector<T>(m + 2, 0)) {}
7
8      void update(int x, int y, T val) {

```

```

9         for (int i = x; i <= n; i += i & -i) {
10             for (int j = y; j <= m; j += j & -j) {
11                 tree[i][j] += val;
12             }
13         }
14     }
15
16     T getPrefix(int x, int y) {
17         if (x <= 0 || y <= 0) return 0;
18         T ret = 0; // change default value
19         for (int i = x; i > 0; i &= i - 1) {
20             for (int j = y; j > 0; j &= j - 1) {
21                 ret += tree[i][j];
22             }
23         }
24         return ret;
25     }
26
27     T getSquare(int xl, int yl, int xr, int yr) { // change operation
28         return getPrefix(xr, yr) + getPrefix(xl - 1, yl - 1) -
29             getPrefix(xr, yl - 1) - getPrefix(xl - 1, yr);
30     }
31 };

```

Segment Tree

```

1 struct info {
2     long long sum = 0;
3     info(long long sum = 0) : sum(sum) {}
4     friend info operator+(const info &l, const info &r) {
5         return {l.sum + r.sum};
6     }
7 };
8
9 template<class info>
10 struct segmentTreeIterative {
11     int n;
12     vector<info> tree;
13
14     explicit segmentTreeIterative(int n) : n(n), tree(n << 1) {}
15
16     template<class U>
17     explicit segmentTreeIterative(const vector<U> &arr) : n(arr.size()),
18         tree(n << 1) {
19         for(int i = 0; i < n; i++) tree[i + n] = info(arr[i]);
20         for(int i = n - 1; i > 0; i--) tree[i] = tree[i << 1] + tree[i << 1 | 1];
21     }
22
23     void set(int i, info v) {
24         for(tree[i += n] = v; i >>= 1; )
25             tree[i] = tree[i << 1] + tree[i << 1 | 1];
26     }
27
28     info get(int l, int r) {
29         info resL, resR;
30         for(l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
31             if(l & 1) resL = resL + tree[l++];

```

```

32         if(r & 1) resR = tree[--r] + resR;
33     }
34     return resL + resR;
35 }
36 };

```

Segment Tree (Recursive)

```

1  struct info {
2      long long sum = 0;
3      info(long long sum = 0) : sum(sum) {}
4      friend info operator+(const info &l, const info &r) {
5          return {l.sum + r.sum};
6      }
7  };
8
9  template<class info>
10 struct dynamicSegmentTree {
11     struct node { int l = 0, r = 0; info v; };
12     vector<node> tr;
13     int n, root = 0;
14
15     explicit dynamicSegmentTree(int n = 1e9, int expectedOps = 1) : n(n), tr(1) {
16         tr.reserve(expectedOps * __lg(n) * 2);
17     }
18
19     info get(int x, int lx, int rx, int l, int r) {
20         if (!x || lx > r || l > rx) return info();
21         if (lx >= l && rx <= r) return tr[x].v;
22         int m = (lx + rx) >> 1;
23         return get(tr[x].l, lx, m, l, r) + get(tr[x].r, m + 1, rx, l, r);
24     }
25
26     int set(int x, int lx, int rx, int i, info val) {
27         if (i < lx || i > rx) return x;
28         if (!x) x = tr.size(), tr.emplace_back();
29         if (lx == rx) return tr[x].v = val, x;
30         int m = (lx + rx) >> 1;
31         tr[x].l = set(tr[x].l, lx, m, i, val);
32         tr[x].r = set(tr[x].r, m + 1, rx, i, val);
33         return tr[x].v = tr[tr[x].l].v + tr[tr[x].r].v, x;
34     }
35
36     void set(int i, info v) { root = set(root, 0, n, i, v); }
37     info get(int l, int r) { return get(root, 0, n, l, r); }
38 };

```

2D Segment Tree

```

1  struct info {
2      long long sum = 0;
3      info(long long sum = 0) : sum(sum) {}
4      friend info operator+(const info &l, const info &r) {
5          return {l.sum + r.sum};
6      }
7  };

```



```

8
9 template<class info>
10 struct segmentTree2d {
11     int nx, ny;
12     vector<vector<info>> tree;
13
14     explicit segmentTree2d(int n, int m) : nx(n), ny(m), tree(nx << 1,
15         vector<info>(ny << 1)) {}
16
17     template<class U>
18     explicit segmentTree2d(const vector<vector<U>> &a) {
19         nx = a.size();
20         ny = nx ? a[0].size() : 0;
21         tree.assign(nx << 1, vector<info>(ny << 1));
22
23         for(int i = 0; i < nx; i++) {
24             for(int j = 0; j < ny; j++) tree[i + nx][j + ny] = info(a[i][j]);
25             for(int y = ny - 1; y > 0; y--)
26                 tree[i + nx][y] = tree[i + nx][y << 1] + tree[i + nx][y << 1 | 1];
27         }
28         for(int x = nx - 1; x > 0; x--)
29             for(int y = 0; y < (ny << 1); y++)
30                 tree[x][y] = tree[x << 1][y] + tree[x << 1 | 1][y];
31     }
32
33     void set(int i, int j, info v) {
34         if (i < 0 || i >= nx || j < 0 || j >= ny) return;
35         int x = i + nx, y = j + ny;
36         tree[x][y] = v;
37         for(int yy = y >> 1; yy > 0; yy >>= 1)
38             tree[x][yy] = tree[x][yy << 1] + tree[x][yy << 1 | 1];
39
40         for(int xx = x >> 1; xx > 0; xx >>= 1) {
41             tree[xx][y] = tree[xx << 1][y] + tree[xx << 1 | 1][y];
42             for(int yy = y >> 1; yy > 0; yy >>= 1)
43                 tree[xx][yy] = tree[xx][yy << 1] + tree[xx][yy << 1 | 1];
44         }
45     }
46
47     info queryY(int nodeX, int y1, int y2) const {
48         info resL, resR;
49         for(int l = y1 + ny, r = y2 + ny + 1; l < r; l >>= 1, r >>= 1) {
50             if (l & 1) resL = resL + tree[nodeX][l++];
51             if (r & 1) resR = tree[nodeX][--r] + resR;
52         }
53         return resL + resR;
54     }
55
56     info get(int x1, int y1, int x2, int y2) {
57         if (!nx || !ny) return info();
58         x1 = max(x1, 0); y1 = max(y1, 0);
59         x2 = min(x2, nx - 1); y2 = min(y2, ny - 1);
60         if (x1 > x2 || y1 > y2) return info();
61
62         info resL, resR;
63         for(int l = x1 + nx, r = x2 + nx + 1; l < r; l >>= 1, r >>= 1) {

```

```

64         if (l & 1) resL = resL + queryY(l++, y1, y2);
65         if (r & 1) resR = queryY(--r, y1, y2) + resR;
66     }
67     return resL + resR;
68 }
69 };

```

Lazy Segment Tree

```

1  struct tag {
2      ll add = 0;
3      void apply(const tag &p) {
4          add += p.add;
5      }
6  };
7
8  struct info {
9      ll sum = 0;
10     info(ll sum = 0) : sum(sum) {}
11     void apply(const tag &lz, int lx, int rx) {
12         sum += 1ll * (rx - lx + 1) * lz.add;
13     }
14     friend info operator+(const info &l, const info &r) {
15         return {l.sum + r.sum};
16     }
17 };
18
19 template<class info, class tag>
20 struct lazySegment {
21     int n;
22     vector<info> tr;
23     vector<tag> lz;
24
25     explicit lazySegment(int _n) : n(_n + 1), tr(4 << __lg(n)), lz(4 << __lg(n)) {}
26     template<class U>
27     explicit lazySegment(const U &arr) : n(arr.size()), tr(4 << __lg(n)),
28         lz(4 << __lg(n)) {
29         build(1, 0, n - 1, arr);
30     }
31
32     void push(int x, int lx, int rx) {
33         if (lx != rx) lz[x << 1].apply(lz[x]), lz[x << 1 | 1].apply(lz[x]);
34         tr[x].apply(lz[x], lx, rx);
35         lz[x] = tag();
36     }
37
38     template<class U>
39     void build(int x, int lx, int rx, const U &arr) {
40         if (lx == rx) return void(tr[x] = arr[lx]);
41         int m = (lx + rx) >> 1;
42         build(x << 1, lx, m, arr), build(x << 1 | 1, m + 1, rx, arr);
43         tr[x] = tr[x << 1] + tr[x << 1 | 1];
44     }
45
46     info get(int x, int lx, int rx, int l, int r) {
47         push(x, lx, rx);

```

```

48         if (lx > r || l > rx) return info();
49         if (lx >= l && rx <= r) return tr[x];
50         int m = (lx + rx) >> 1;
51         return get(x << 1, lx, m, l, r) + get(x << 1 | 1, m + 1, rx, l, r);
52     }
53
54     void set(int x, int lx, int rx, int i, info val) {
55         push(x, lx, rx);
56         if (i < lx || i > rx) return;
57         if (lx == rx) return void(tr[x] = val);
58         int m = (lx + rx) >> 1;
59         set(x << 1, lx, m, i, val), set(x << 1 | 1, m + 1, rx, i, val);
60         tr[x] = tr[x << 1] + tr[x << 1 | 1];
61     }
62
63     void applyRange(int x, int lx, int rx, int l, int r, tag v) {
64         push(x, lx, rx);
65         if (lx > r || l > rx) return;
66         if (lx >= l && rx <= r) return lz[x] = v, push(x, lx, rx);
67         int m = (lx + rx) >> 1;
68         applyRange(x << 1, lx, m, l, r, v),
69         applyRange(x << 1 | 1, m + 1, rx, l, r, v);
70         tr[x] = tr[x << 1] + tr[x << 1 | 1];
71     }
72
73     void set(int i, info v) { set(1, 0, n - 1, i, v); }
74     void applyRange(int l, int r, tag v) { applyRange(1, 0, n - 1, l, r, v); }
75     info get(int l, int r) { return get(1, 0, n - 1, l, r); }
76 };

```

Dynamic Lazy Segment Tree

```

1  template<class info, class tag>
2  struct dynamicLazySegmentTree {
3      struct node { int l = 0, r = 0; info v; tag t; };
4      vector<node> tr;
5      int n, root = 0;
6
7      explicit dynamicLazySegmentTree(int n = 1e9) : n(n), tr(1) {}
8
9      void push(int x, int lx, int rx) {
10         if (lx == rx) return void(tr[x].t = tag());
11         if (!tr[x].l) tr[x].l = tr.size(), tr.emplace_back();
12         if (!tr[x].r) tr[x].r = tr.size(), tr.emplace_back();
13
14         int m = (lx + rx) >> 1, l = tr[x].l, r = tr[x].r;
15         tr[l].t.apply(tr[x].t), tr[l].v.apply(tr[x].t, lx, m);
16         tr[r].t.apply(tr[x].t), tr[r].v.apply(tr[x].t, m + 1, rx);
17         tr[x].t = tag();
18     }
19
20     int set(int x, int lx, int rx, int i, info val) {
21         if (i < lx || i > rx) return x;
22         if (!x) x = tr.size(), tr.emplace_back();
23         if (lx == rx) {

```

```

24         tr[x].v = val, tr[x].t = tag();
25         return x;
26     }
27     push(x, lx, rx);
28     int m = (lx + rx) >> 1;
29     tr[x].l = set(tr[x].l, lx, m, i, val);
30     tr[x].r = set(tr[x].r, m + 1, rx, i, val);
31     return tr[x].v = tr[tr[x].l].v + tr[tr[x].r].v, x;
32 }
33
34 int applyRange(int x, int lx, int rx, int l, int r, tag val) {
35     if (lx > r || l > rx) return x;
36     if (!x) x = tr.size(), tr.emplace_back();
37     if (lx >= l && rx <= r) {
38         tr[x].t.apply(val);
39         tr[x].v.apply(val, lx, rx);
40         return x;
41     }
42     push(x, lx, rx);
43     int m = (lx + rx) >> 1;
44     tr[x].l = applyRange(tr[x].l, lx, m, l, r, val);
45     tr[x].r = applyRange(tr[x].r, m + 1, rx, l, r, val);
46     return tr[x].v = tr[tr[x].l].v + tr[tr[x].r].v, x;
47 }
48
49 info get(int x, int lx, int rx, int l, int r) {
50     if (!x || lx > r || l > rx) return info();
51     if (lx >= l && rx <= r) return tr[x].v;
52     push(x, lx, rx);
53     int m = (lx + rx) >> 1;
54     return get(tr[x].l, lx, m, l, r) + get(tr[x].r, m + 1, rx, l, r);
55 }
56
57 void set(int i, info v) { root = set(root, 0, n, i, v); }
58 void applyRange(int l, int r, tag v) { root = applyRange(root, 0, n, l, r, v); }
59 info get(int l, int r) { return get(root, 0, n, l, r); }
60 };

```

Segment Tree Beats

```

1  struct segtreebeats {
2      static const ll INF = 2e18, UNSET = LLONG_MIN;
3
4      struct Node {
5          ll sum, mx1, mx2, mxc, mn1, mn2, mnc, d_gcd, lz_add, lz_set;
6      };
7
8      int sz;
9      vector<Node> tree;
10
11     Node leaf(ll v) { return {v, v, -INF, 1, v, INF, 1, 0, 0, UNSET}; }
12
13     segtreebeats(int n, const vector<ll> &a) {
14         for (sz = 1; sz < n; sz <= 1);
15         tree.assign(sz << 1, leaf(0));
16         build(a, n, 0, 0, sz - 1);

```

```

17     }
18
19     Node merge(const Node &L, const Node &R) {
20         Node U = {L.sum + R.sum, 0, 0, 0, 0, 0, 0, gcd(L.d_gcd, R.d_gcd), 0, UNSET};
21
22         if (L.mx1 == R.mx1) U.mx1 = L.mx1, U.mx2 = max(L.mx2, R.mx2), U.mxc = L.mxc + R.mxc;
23         else if (L.mx1 > R.mx1) U.mx1 = L.mx1, U.mx2 = max(L.mx2, R.mx1), U.mxc = L.mxc;
24         else U.mx1 = R.mx1, U.mx2 = max(L.mx1, R.mx2), U.mxc = R.mxc;
25
26         if (L.mn1 == R.mn1) U.mn1 = L.mn1, U.mn2 = min(L.mn2, R.mn2), U.mnc = L.mnc + R.mnc;
27         else if (L.mn1 < R.mn1) U.mn1 = L.mn1, U.mn2 = min(L.mn2, R.mn1), U.mnc = L.mnc;
28         else U.mn1 = R.mn1, U.mn2 = min(L.mn1, R.mn2), U.mnc = R.mnc;
29
30         ll aL = L.mx2, aR = R.mx2;
31         if (aL != -INF && aL != L.mn1 && aR != -INF && aR != R.mn1)
32             U.d_gcd = gcd(U.d_gcd, abs(aL - aR));
33
34         ll any = UNSET;
35         if (aL != -INF && aL != L.mn1) any = aL;
36         else if (aR != -INF && aR != R.mn1) any = aR;
37
38         for (ll val : {L.mn1, L.mx1, R.mn1, R.mx1}) {
39             if (val != U.mn1 && val != U.mx1) {
40                 if (any != UNSET) U.d_gcd = gcd(U.d_gcd, abs(val - any));
41                 else any = val;
42             }
43         }
44         return U;
45     }
46
47     void apply_set(int x, int lx, int rx, ll v) {
48         ll len = rx - lx + 1;
49         tree[x] = {len * v, v, -INF, len, v, INF, len, 0, 0, v};
50     }
51
52     void apply_add(int x, int lx, int rx, ll v) {
53         if (!v) return;
54         auto &nd = tree[x];
55         if (nd.lz_set != UNSET) return apply_set(x, lx, rx, nd.lz_set + v);
56         if (nd.mx1 == nd.mn1) return apply_set(x, lx, rx, nd.mn1 + v);
57         ll len = rx - lx + 1;
58         nd.sum += len * v;
59         nd.mx1 += v; if (nd.mx2 != -INF) nd.mx2 += v;
60         nd.mn1 += v; if (nd.mn2 != INF) nd.mn2 += v;
61         nd.lz_add += v;
62     }
63
64     void apply_chmin(int x, int lx, int rx, ll v) {
65         auto &nd = tree[x];
66         if (nd.mx1 <= v) return;
67         if (nd.mn1 >= v) return apply_set(x, lx, rx, v);
68         if (nd.mn2 == nd.mx1) nd.mn2 = v;
69         nd.sum -= (nd.mx1 - v) * nd.mxc;
70         nd.mx1 = v;
71     }
72

```

```

73 void apply_chmax(int x, int lx, int rx, ll v) {
74     auto &nd = tree[x];
75     if (nd.mn1 >= v) return;
76     if (nd.mx1 <= v) return apply_set(x, lx, rx, v);
77     if (nd.mx2 == nd.mn1) nd.mx2 = v;
78     nd.sum += (v - nd.mn1) * nd.mnc;
79     nd.mn1 = v;
80 }
81
82 void push(int x, int lx, int rx) {
83     if (lx == rx) return tree[x].lz_add = 0, tree[x].lz_set = UNSET, void();
84     int m = (lx + rx) >> 1, lc = x * 2 + 1, rc = x * 2 + 2;
85
86     if (tree[x].lz_set != UNSET) {
87         apply_set(lc, lx, m, tree[x].lz_set);
88         apply_set(rc, m + 1, rx, tree[x].lz_set);
89         tree[x].lz_set = UNSET;
90     }
91     if (tree[x].lz_add) {
92         apply_add(lc, lx, m, tree[x].lz_add);
93         apply_add(rc, m + 1, rx, tree[x].lz_add);
94         tree[x].lz_add = 0;
95     }
96     if (tree[lc].mx1 > tree[x].mx1) apply_chmin(lc, lx, m, tree[x].mx1);
97     if (tree[rc].mx1 > tree[x].mx1) apply_chmin(rc, m + 1, rx, tree[x].mx1);
98     if (tree[lc].mn1 < tree[x].mn1) apply_chmax(lc, lx, m, tree[x].mn1);
99     if (tree[rc].mn1 < tree[x].mn1) apply_chmax(rc, m + 1, rx, tree[x].mn1);
100 }
101
102 void build(const vector<ll> &a, int n, int x, int lx, int rx) {
103     if (lx == rx) return void(tree[x] = (lx < n) ? leaf(a[lx]) : leaf(0));
104     int m = (lx + rx) >> 1;
105     build(a, n, x * 2 + 1, lx, m);
106     build(a, n, x * 2 + 2, m + 1, rx);
107     tree[x] = merge(tree[x * 2 + 1], tree[x * 2 + 2]);
108 }
109
110 void chmin(int l, int r, ll v, int x, int lx, int rx) {
111     if (lx > r || rx < l || tree[x].mx1 <= v) return;
112     if (lx >= l && rx <= r && tree[x].mx2 < v) return apply_chmin(x, lx, rx, v);
113     push(x, lx, rx);
114     int m = (lx + rx) >> 1;
115     chmin(l, r, v, x * 2 + 1, lx, m);
116     chmin(l, r, v, x * 2 + 2, m + 1, rx);
117     tree[x] = merge(tree[x * 2 + 1], tree[x * 2 + 2]);
118 }
119
120 void chmax(int l, int r, ll v, int x, int lx, int rx) {
121     if (lx > r || rx < l || tree[x].mn1 >= v) return;
122     if (lx >= l && rx <= r && tree[x].mn2 > v) return apply_chmax(x, lx, rx, v);
123     push(x, lx, rx);
124     int m = (lx + rx) >> 1;
125     chmax(l, r, v, x * 2 + 1, lx, m);
126     chmax(l, r, v, x * 2 + 2, m + 1, rx);
127     tree[x] = merge(tree[x * 2 + 1], tree[x * 2 + 2]);

```

```

128     }
129
130 void assign(int l, int r, ll v, int x, int lx, int rx) {
131     if (lx > r || rx < l) return;
132     if (lx >= l && rx <= r) return apply_set(x, lx, rx, v);
133     push(x, lx, rx);
134     int m = (lx + rx) >> 1;
135     assign(l, r, v, x * 2 + 1, lx, m);
136     assign(l, r, v, x * 2 + 2, m + 1, rx);
137     tree[x] = merge(tree[x * 2 + 1], tree[x * 2 + 2]);
138 }
139
140 void add(int l, int r, ll v, int x, int lx, int rx) {
141     if (lx > r || rx < l) return;
142     if (lx >= l && rx <= r) return apply_add(x, lx, rx, v);
143     push(x, lx, rx);
144     int m = (lx + rx) >> 1;
145     add(l, r, v, x * 2 + 1, lx, m);
146     add(l, r, v, x * 2 + 2, m + 1, rx);
147     tree[x] = merge(tree[x * 2 + 1], tree[x * 2 + 2]);
148 }
149
150 ll sum(int l, int r, int x, int lx, int rx) {
151     if (lx > r || rx < l) return 0;
152     if (lx >= l && rx <= r) return tree[x].sum;
153     push(x, lx, rx);
154     int m = (lx + rx) >> 1;
155     return sum(l, r, x * 2 + 1, lx, m) + sum(l, r, x * 2 + 2, m + 1, rx);
156 }
157
158 ll qmin(int l, int r, int x, int lx, int rx) {
159     if (lx > r || rx < l) return INF;
160     if (lx >= l && rx <= r) return tree[x].mn1;
161     push(x, lx, rx);
162     int m = (lx + rx) >> 1;
163     return min(qmin(l, r, x * 2 + 1, lx, m), qmin(l, r, x * 2 + 2, m + 1, rx));
164 }
165
166 ll qmax(int l, int r, int x, int lx, int rx) {
167     if (lx > r || rx < l) return -INF;
168     if (lx >= l && rx <= r) return tree[x].mx1;
169     push(x, lx, rx);
170     int m = (lx + rx) >> 1;
171     return max(qmax(l, r, x * 2 + 1, lx, m), qmax(l, r, x * 2 + 2, m + 1, rx));
172 }
173
174 ll qgcd(int l, int r, int x, int lx, int rx) {
175     if (lx > r || rx < l) return 0;
176     if (lx >= l && rx <= r) {
177         ll ans = gcd(tree[x].d_gcd, abs(tree[x].mx1));
178         if (tree[x].mx2 != -INF) ans = gcd(ans, abs(tree[x].mx2 - tree[x].mx1));
179         if (tree[x].mn2 != INF) ans = gcd(ans, abs(tree[x].mn2 - tree[x].mn1));
180         return ans;
181     }
182     push(x, lx, rx);

```

```

183         int m = (lx + rx) >> 1;
184         return gcd(qgcd(l, r, x * 2 + 1, lx, m), qgcd(l, r, x * 2 + 2, m + 1, rx));
185     }
186
187     // Public Wrappers
188     void chmin(int l, int r, ll v) { chmin(l, r, v, 0, 0, sz - 1); }
189     void chmax(int l, int r, ll v) { chmax(l, r, v, 0, 0, sz - 1); }
190     void assign(int l, int r, ll v) { assign(l, r, v, 0, 0, sz - 1); }
191     void add(int l, int r, ll v) { add(l, r, v, 0, 0, sz - 1); }
192     ll qsum(int l, int r) { return sum(l, r, 0, 0, sz - 1); }
193     ll qmin(int l, int r) { return qmin(l, r, 0, 0, sz - 1); }
194     ll qmax(int l, int r) { return qmax(l, r, 0, 0, sz - 1); }
195     ll qgcd(int l, int r) { return qgcd(l, r, 0, 0, sz - 1); }
196 };

```

Dynamic Persistent Segment Tree

```

1  template<class info>
2  struct DPSGT {
3      struct node { int l = 0, r = 0; info v; };
4      vector<node> tr;
5      int n;
6
7      explicit DPSGT(int n = 1e9, int expectedOps = 1) : n(n), tr(1) {
8          tr.reserve(expectedOps * __lg(n) * 2);
9      }
10
11     info get(int x, int lx, int rx, int l, int r) {
12         if (!x || lx > r || l > rx) return info();
13         if (lx >= l && rx <= r) return tr[x].v;
14         int m = (lx + rx) >> 1;
15         return get(tr[x].l, lx, m, l, r) + get(tr[x].r, m + 1, rx, l, r);
16     }
17
18     int set(int x, int lx, int rx, int i, info val) {
19         if (i < lx || i > rx) return x;
20         int nx = tr.size();
21         tr.push_back(tr[x]);
22         if (lx == rx) return tr[nx].v = val, nx;
23         int m = (lx + rx) >> 1;
24         tr[nx].l = set(tr[nx].l, lx, m, i, val);
25         tr[nx].r = set(tr[nx].r, m + 1, rx, i, val);
26         return tr[nx].v = tr[tr[nx].l].v + tr[tr[nx].r].v, nx;
27     }
28
29     int set(int root, int i, info v) { return set(root, 0, n, i, v); }
30     info get(int root, int l, int r) { return get(root, 0, n, l, r); }
31 };

```

Wavelet Tree

```

1  struct WaveletTree {
2      int lo, hi;
3      WaveletTree *lc = 0, *rc = 0;
4      vector<int> b;

```



```

5
6 template<class It>
7 WaveletTree(It from, It to, int x, int y) : lo(x), hi(y) {
8     if (from >= to || lo == hi) return;
9     int mid = lo + (hi - lo) / 2;
10
11     b.reserve(distance(from, to) + 1);
12     b.push_back(0);
13     for (auto it = from; it != to; ++it) {
14         b.push_back(b.back() + (*it <= mid));
15     }
16
17     auto pivot = stable_partition(from, to, [mid](int val)
18         { return val <= mid; });
19     lc = new WaveletTree(from, pivot, lo, mid);
20     rc = new WaveletTree(pivot, to, mid + 1, hi);
21 }
22
23 // the passed vector is changed, pass a copy in the main
24 // Takes a vector of integers, and the min/max values in it.
25 // The vector should be 0-indexed but queries works as 1-based
26 WaveletTree(vector<int>& a, int x, int y) :
27 WaveletTree(a.begin(), a.end(), x, y) {}
28
29 ~WaveletTree() { delete lc; delete rc; }
30
31 int kth(int l, int r, int k) {
32     if (l > r) return 0;
33     if (lo == hi) return lo;
34     int in_left = b[r] - b[l - 1], lb = b[l - 1], rb = b[r];
35     if (k <= in_left) return lc->kth(lb + 1, rb, k);
36     return rc->kth(l - lb, r - rb, k - in_left);
37 }
38
39 int LTE(int l, int r, int k) {
40     if (l > r || k < lo) return 0;
41     if (hi <= k) return r - l + 1;
42     int lb = b[l - 1], rb = b[r];
43     return lc->LTE(lb + 1, rb, k) + rc->LTE(l - lb, r - rb, k);
44 }
45
46 int count(int l, int r, int k) {
47     if (l > r || k < lo || k > hi) return 0;
48     if (lo == hi) return r - l + 1;
49     int mid = lo + (hi - lo) / 2, lb = b[l - 1], rb = b[r];
50     if (k <= mid) return lc->count(lb + 1, rb, k);
51     return rc->count(l - lb, r - rb, k);
52 }
53 };

```

Implicit Treap

```

1 // You must manually track the root of your tree in your main function:
2 //     Treap tree;
3 //     int root = 0;
4 // 1. Insert val at index i:

```

```

5 //      int l, r;
6 //      tree.split(root, i, l, r);
7 //      int mid = tree.new_node(val);
8 //      root = tree.merge(tree.merge(l, mid), r);
9 //
10 // 2. Delete the element at index i:
11 //      auto [l, mid, r] = tree.split(root, i, i);
12 //      root = tree.merge(l, r); // mid is safely dropped and ignored
13 //
14 // 3. Range Sum for subarray [L, R]:
15 //      auto [l, mid, r] = tree.split(root, L, R);
16 //      long long ans = tree.tr[mid].sum;
17 //      root = tree.merge(tree.merge(l, mid), r); // ALWAYS merge back!
18 //
19 // 4. Range Reverse for subarray [L, R]:
20 //      auto [l, mid, r] = tree.split(root, L, R);
21 //      tree.tr[mid].rev ^= 1; // apply the lazy tag
22 //      root = tree.merge(tree.merge(l, mid), r); // ALWAYS merge back!
23 // -----
24
25 mt19937 rnd(time(nullptr));
26 struct Treap {
27     struct node {
28         uint32_t pri = rnd();
29         int sz = 1, l = 0, r = 0, val{};
30         int64_t sum{};
31         bool rev = false;
32         node(int x) : sum(val = x) { }
33     };
34     vector<node> tr;
35
36     // tr[0] acts as a dummy/null node to avoid segmentation faults
37     Treap() : tr(1, 0) { tr[0].sz = 0; }
38
39     inline void pull(int x) {
40         tr[x].sz = tr[tr[x].l].sz + tr[tr[x].r].sz + 1;
41         tr[x].sum = tr[tr[x].l].sum + tr[tr[x].r].sum + tr[x].val;
42     }
43
44     inline void push(int x) {
45         if(tr[x].rev) {
46             tr[tr[x].l].rev ^= 1, tr[tr[x].r].rev ^= 1;
47             swap(tr[x].l, tr[x].r);
48             tr[x].rev = false;
49         }
50     }
51
52     // Returns the index of the newly created node
53     inline int new_node(int val) {
54         tr.emplace_back(val);
55         return int(tr.size()) - 1;
56     }
57
58     // Concatenates treap rx to the right of lx. Returns the new root.
59     int merge(int lx, int rx) {
60         if(!lx || !rx) return rx + lx;
61         push(lx), push(rx);

```

```

62         if(tr[lx].pri < tr[rx].pri)
63             return tr[rx].l = merge(lx, tr[rx].l), pull(rx), rx;
64         return tr[lx].r = merge(tr[lx].r, rx), pull(lx), lx;
65     }
66
67     // Splits treap x into lx (first k elements) and rx (the rest).
68     void split(int x, int k, int &lx, int &rx) {
69         if(!x) return lx = rx = 0, void();
70         push(x);
71         if(k <= tr[tr[x].l].sz) split(tr[x].l, k, lx, tr[x].l), pull(rx = x);
72         else split(tr[x].r, k - tr[tr[x].l].sz - 1, tr[x].r, rx), pull(lx = x);
73     }
74
75     // Helper to extract a specific subarray [l, r] into the middle partition
76     array<int, 3> split(int x, int l, int r) {
77         int a, b, c;
78         split(x, r + 1, b, c);
79         split(b, l, a, b);
80         return {a, b, c};
81     }
82 };

```

Sparse Table

```

1  struct ST {
2      static const int K = 18, N = 2e5 + 5;
3      int st[K][N];
4
5      void build(int n, const vector<int>& a) {
6          for(int i = 0; i < n; i++) st[0][i] = a[i];
7          for(int k = 1; k < K; k++)
8              for(int i = 0; i + (1 << k) <= n; i++)
9                  st[k][i] = min(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
10     }
11
12     int query(int l, int r) {
13         int k = __lg(r - l + 1);
14         return min(st[k][l], st[k][r - (1 << k) + 1]);
15     }
16 };
17
18
19 template<class T, class F>
20 struct sparse {
21     int n, Log;
22     vector<vector<T>> table;
23     F merge;
24     T id;
25
26     sparse(const vector<T>& arr, F merge, T id = T()) :
27         n(arr.size()), Log(__lg(n) + 1), table(Log, vector<T>(n)),
28         merge(merge), id(id) {
29         table[0] = arr;
30         for (int l = 1; l < Log; l++) {
31             for (int i = 0; i + (1 << (l - 1)) < n; i++) {
32                 table[l][i] = merge(table[l - 1][i],

```

```

33         table[l - 1][i + (1 << (l - 1))]);
34     }
35 }
36 }
37
38 T query(int l, int r) {
39     if (l > r) return id;
40     int len = __lg(r - l + 1);
41     return merge(table[len][l], table[len][r - (1 << len) + 1]);
42 }
43
44 T query_log(int l, int r) {
45     T res = id;
46     bool first = true;
47     for (int j = Log - 1; j >= 0; j--) {
48         if (1 << j <= r - l + 1) {
49             res = first ? table[j][l] : merge(res, table[j][l]);
50             first = false;
51             l += 1 << j;
52         }
53     }
54     return res;
55 }
56 };
57 // sparse st(a, [](int x, int y) { return min(x, y); }, INT_MAX);

```

2D Sparse Table

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  template<class T, class F>
5  struct sparse2d {
6      int n, m, Kx, Ky;
7      vector<int> lgx, lgy;
8      vector<vector<vector<vector<T>>>> st;
9      F merge;
10
11      sparse2d(const vector<vector<T>>& a, F merge) : merge(merge) {
12          n = (int)a.size();
13          m = (int)a[0].size();
14
15          lgx.resize(n + 1);
16          lgy.resize(m + 1);
17          for (int i = 2; i <= n; ++i) lgx[i] = lgx[i / 2] + 1;
18          for (int j = 2; j <= m; ++j) lgy[j] = lgy[j / 2] + 1;
19
20          Kx = lgx[n];
21          Ky = lgy[m];
22
23          st.assign(Kx + 1, vector<vector<vector<T>>>(Ky + 1));
24
25          st[0][0] = a;
26
27          // Build along columns for kx = 0
28          for (int ky = 1; ky <= Ky; ++ky) {
29              int len = 1 << ky;

```

```

30         int half = len >> 1;
31         st[0][ky].assign(n, vector<T>(m - len + 1));
32         for (int i = 0; i < n; ++i) {
33             for (int j = 0; j + len <= m; ++j) {
34                 st[0][ky][i][j] = merge(st[0][ky - 1][i][j],
35                                         st[0][ky - 1][i][j + half]);
36             }
37         }
38     }
39
40     // Build along rows for all ky
41     for (int kx = 1; kx <= Kx; ++kx) {
42         int lenx = 1 << kx;
43         int halfx = lenx >> 1;
44         for (int ky = 0; ky <= Ky; ++ky) {
45             int leny = 1 << ky;
46             st[kx][ky].assign(n - lenx + 1, vector<T>(m - leny + 1));
47             for (int i = 0; i + lenx <= n; ++i) {
48                 for (int j = 0; j + leny <= m; ++j) {
49                     st[kx][ky][i][j] = merge(st[kx - 1][ky][i][j],
50                                             st[kx - 1][ky][i + halfx][j]);
51                 }
52             }
53         }
54     }
55 }
56
57 // inclusive coordinates: [x1..x2], [y1..y2]
58 T query(int x1, int y1, int x2, int y2) const {
59     int kx = lgx[x2 - x1 + 1];
60     int ky = lgy[y2 - y1 + 1];
61     int dx = 1 << kx;
62     int dy = 1 << ky;
63
64     const T& a = st[kx][ky][x1][y1];
65     const T& b = st[kx][ky][x2 - dx + 1][y1];
66     const T& c = st[kx][ky][x1][y2 - dy + 1];
67     const T& d = st[kx][ky][x2 - dx + 1][y2 - dy + 1];
68
69     return merge(merge(a, b), merge(c, d));
70 }
71 };

```

DSU

```

1 struct DSU {
2     vector<int> p, sz;
3     int n, comps;
4     DSU(int _n = 0) { init(_n); }
5     void init(int _n) {
6         n = _n + 10; comps = _n;
7         p.resize(n); sz.assign(n, 1);
8         iota(p.begin(), p.end(), 0);
9     }
10    int find(int u) { return u == p[u] ? u : p[u] = find(p[u]); }
11    bool unite(int u, int v) {

```

```

12     u = find(u), v = find(v);
13     if (u == v) return 0;
14     if (sz[u] < sz[v]) swap(u, v);
15     p[v] = u; sz[u] += sz[v]; comps--;
16     return 1;
17 }
18 bool same(int u, int v) { return find(u) == find(v); }
19 int size(int u) { return sz[find(u)]; }
20 int size() { return comps; }
21 };

```

Persistent DSU

```

1 struct persistent_DSU {
2     vector<vector<pair<int, int> > > par;
3     persistent_DSU(int n) : par(n + 1, {{-1, 0}}) { }
4
5     int time = 0;
6     bool merge(int u, int v) {
7         ++time;
8         if ((u = root(u, time)) == (v = root(v, time)))
9             return 0;
10        if (par[u].back().first > par[v].back().first)
11            swap(u, v);
12        par[u].push_back({par[u].back().first + par[v].back().first, time});
13        par[v].push_back({u, time}); // par[v] = u
14        return 1;
15    }
16
17    bool same(int u, int v, int t) {
18        return root(u, t) == root(v, t);
19    }
20
21    int root(int u, int t) {
22        // root of u at time t
23        if (par[u].back().first >= 0 && par[u].back().second <= t)
24            return root(par[u].back().first, t);
25        return u;
26    }
27
28    int size(int u, int t) { // O(log)
29        // size of the component of u at time t
30        u = root(u, t);
31        int l = 0, r = (int) par[u].size() - 1, ans = 0;
32        while (l <= r) {
33            int mid = l + r >> 1;
34            if (par[u][mid].second <= t)
35                ans = mid, l = mid + 1;
36            else
37                r = mid - 1;
38        }
39        return -par[u][ans].first;
40    }
41 };

```

Bipartite DSU

```
1 // Maintains whether each component is bipartite
2 struct BipartiteDSU {
3     vector<int> sz, bipartite;
4     vector<pair<int, int> > par;
5
6     BipartiteDSU(int n) : par(n), sz(n, 1), bipartite(n) {
7         for (int i = 0; i < n; ++i) {
8             par[i] = {i, 0};
9         }
10    }
11
12    pair<int, int> find(int u) {
13        if (u == par[u].first) return {u, 0};
14        int parity = par[u].second;
15        par[u] = find(par[u].first);
16        par[u].second ^= parity;
17        return par[u];
18    }
19
20    bool same(int x, int y) {
21        return find(x).first == find(y).first;
22    }
23
24    bool join(int u, int v) {
25        pair<int, int> pu = find(u);
26        pair<int, int> pv = find(v);
27        u = pu.first;
28        v = pv.first;
29        int x = pu.second, y = pv.second;
30        if (u == v) {
31            if (x == y) bipartite[u] = false;
32            return false;
33        }
34        if (sz[u] < sz[v]) swap(u, v);
35        par[v] = {u, x ^ y ^ 1};
36        bipartite[u] ^= bipartite[v];
37        sz[u] += sz[v];
38        return true;
39    }
40
41    int size(int x) { return sz[find(x).first]; }
42 };
```

Monotonic Stack / Queue

```
1 template<class T>
2 struct Mono_stack {
3     stack<pair<T, T> > st;
4
5     void push(const T &val) {
6         if (st.empty()) st.emplace(val, val);
7         else st.emplace(val, max(val, st.top().second));
8     }
9
10    void pop() { st.pop(); }
```

```

11     bool empty() { return st.empty(); }
12     int size() { return st.size(); }
13     T top() { return st.top().first; }
14     T max() { return st.top().second; }
15 };
16
17 template<class T>
18 struct Mono_queue {
19     Mono_stack<T> pop_st, push_st;
20
21     void push(const T &val) { push_st.push(val); }
22
23     void move() {
24         if (pop_st.size()) return;
25         while (!push_st.empty()) pop_st.push(push_st.top()), push_st.pop();
26     }
27
28     void pop() { move(); pop_st.pop(); }
29     bool empty() { return pop_st.empty() && push_st.empty(); }
30     int size() { return pop_st.size() + push_st.size(); }
31     T top() { move(); return pop_st.top(); }
32
33     T max() {
34         if (pop_st.empty()) return push_st.max();
35         if (push_st.empty()) return pop_st.max();
36         return max(push_st.max(), pop_st.max());
37     }
38 };

```

Ordered Data Structures (pb_ds)

```

1  #include <ext/pb_ds/assoc_container.hpp>
2  #include <ext/pb_ds/tree_policy.hpp>
3  using namespace __gnu_pbds;
4  template <typename T> using ordered_set = tree<T, null_type, less<T>,
5      rb_tree_tag, tree_order_statistics_node_update>;
6  template <typename T, typename R> using ordered_map = tree<T, R, less<T>,
7      rb_tree_tag, tree_order_statistics_node_update>;

```

BucketList

```

1  template<class T>
2  struct BucketList {
3      int siz = 0;
4      vector<vector<T> > a;
5      static constexpr int SPLIT_RATIO = 28;
6
7      void insert(int i, T x) {
8          if (siz == 0) {
9              siz = 1;
10             a = {{x}};
11             return;
12         }
13         siz++;
14         for (ll bi = 0; bi < size(a); bi++) {
15             auto &bucket = a[bi];

```



```

16         if (i <= size(bucket)) {
17             bucket.insert(begin(bucket) + i, x);
18             if (size(bucket) > size(a) * SPLIT_RATIO) {
19                 auto L = end(bucket) - size(bucket) / 2, R = end(bucket);
20                 a.emplace(begin(a) + bi + 1, L, R);
21                 // bucket might be broken
22                 a[bi].erase(L, R);
23             }
24             return;
25         }
26         i -= size(bucket);
27     }
28 }
29
30 void erase(int i) {
31     siz--;
32     for (int bi = 0; bi < size(a); bi++) {
33         auto& bucket = a[bi];
34         if (i < size(bucket)) {
35             bucket.erase(begin(bucket) + i);
36             if (bucket.empty()) a.erase(begin(a) + bi);
37             return;
38         }
39         i -= size(bucket);
40     }
41 }
42
43 T& access(int i) {
44     for (auto& bucket : a) {
45         if (i < size(bucket)) return bucket[i];
46         i -= size(bucket);
47     }
48     assert(false);
49 }
50 };

```

Full Dynamic Array

```

1  mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
2  template<typename T>
3  struct DynArray {
4      struct Node {
5          T val; int pri, sz;
6          Node *l, *r;
7          Node(const T& v) : val(v), pri(rng()), sz(1), l(0), r(0) {}
8      };
9
10     Node* root = 0;
11
12     static int sz(Node* t) { return t ? t->sz : 0; }
13     static void pull(Node* t) { if (t) t->sz = 1 + sz(t->l) + sz(t->r); }
14
15     static void split(Node* t, int k, Node*& l, Node*& r) {
16         if (!t) { l = r = 0; return; }
17         if (sz(t->l) < k) {
18             split(t->r, k - sz(t->l) - 1, t->r, r);

```

```

19         l = t;
20     } else {
21         split(t->l, k, l, t->l);
22         r = t;
23     }
24     pull(t);
25 }
26
27 static void merge(Node*& t, Node* l, Node* r) {
28     if (!l || !r) { t = l ? l : r; return; }
29     if (l->pri > r->pri) {
30         merge(l->r, l->r, r);
31         t = l;
32     } else {
33         merge(r->l, l, r->l);
34         t = r;
35     }
36     pull(t);
37 }
38
39 void insert(int pos, const T& val) {
40     Node *l, *r;
41     split(root, pos, l, r);
42     Node* nd = new Node(val);
43     merge(l, l, nd);
44     merge(root, l, r);
45 }
46
47 void erase(int pos) {
48     Node *l, *m, *r;
49     split(root, pos, l, m);
50     split(m, 1, m, r);
51     delete m;
52     merge(root, l, r);
53 }
54
55 T& access(int pos) {
56     Node* t = root;
57     while (true) {
58         int left = sz(t->l);
59         if (pos < left) t = t->l;
60         else if (pos == left) return t->val;
61         else { pos -= left + 1; t = t->r; }
62     }
63 }
64
65 int size() { return sz(root); }
66 };

```

Count below

```

1 // counts how many pairs that sum <= limit
2 template <typename T>
3 ll count_below(vector<T>& v, int sz, ll Limit){
4     // unique pairs such that v[i]+v[j] <= limit
5     assert(is_sorted(v.begin(), v.end()));

```

```

6     ll total = 0;
7     for (int l = 0, r = sz-1; l < r; l++){
8         while(r > l && v[l] + v[r] > Limit)
9             r--;
10        total += max(0, r-l);
11    }
12    return total;
13 }

```

3. Graphs and Trees

Bellman Ford

```
1 // Computes Single-Source Shortest Paths (SSSP) on graphs that contain NEGATIVE edge weights.
2 // It explicitly detects negative weight cycles reachable from the source vertex.
3 // - Average Time Complexity:  $O(E)$  where  $E$  is the number of edges.
4 // - Worst Case Time Complexity:  $O(V * E)$ .
5 // vector<int> dist;
6 // bool ok = spfa(0, adj, dist);
7
8 const int INF = 1e9;
9 bool spfa(int s, const vector<vector<pair<int, int>>>& adj, vector<int>& d) {
10     int n = adj.size();
11     d.assign(n, INF);
12     vector<int> cnt(n, 0);
13     vector<int> inqueue(n, 0);
14     queue<int> q;
15
16     d[s] = 0;
17     q.push(s);
18     inqueue[s] = 1;
19
20     while (!q.empty()) {
21         int v = q.front();
22         q.pop();
23         inqueue[v] = 0;
24         for (const auto& [to, len] : adj[v]) {
25             if (d[v] + len < d[to]) {
26                 d[to] = d[v] + len;
27                 if (!inqueue[to]) {
28                     q.push(to);
29                     inqueue[to] = 1;
30                     if (++cnt[to] > n) return false; // negative cycle
31                 }
32             }
33         }
34     }
35     return true;
36 }
```

Floyd Tricks

```
1 void TransitiveClosure(int n, vector<vector<int>>& adj) {
2     // 0 = disconnected, 1 = connected
3     for (int k = 0; k < n; ++k)
4         for (int i = 0; i < n; ++i)
5             for (int j = 0; j < n; ++j)
6                 adj[i][j] |= (adj[i][k] & adj[k][j]);
7 }
8
9 void minimax(int n, vector<vector<int>>& adj) {
10     // Path such that max value on road is minimized
11     for (int k = 0; k < n; ++k)
12         for (int i = 0; i < n; ++i)
13             for (int j = 0; j < n; ++j)
```

```

14         adj[i][j] = min(adj[i][j], max(adj[i][k], adj[k][j]));
15     }
16
17 void maximin(int n, vector<vector<int>>& adj) {
18     // Path such that min value on road is maximized
19     for (int k = 0; k < n; ++k)
20         for (int i = 0; i < n; ++i)
21             for (int j = 0; j < n; ++j)
22                 adj[i][j] = max(adj[i][j], min(adj[i][k], adj[k][j]));
23 }
24
25 void longestPathDAG(int n, vector<vector<int>>& adj) {
26     // Only works for DAGs (no positive cycles)
27     for (int k = 0; k < n; ++k)
28         for (int i = 0; i < n; ++i)
29             for (int j = 0; j < n; ++j)
30                 adj[i][j] = max(adj[i][j], max(adj[i][k], adj[k][j]));
31 }
32
33 void countPaths(int n, vector<vector<int>>& adj) {
34     // Floyd-Warshall for counting number of paths
35     for (int k = 0; k < n; ++k)
36         for (int i = 0; i < n; ++i)
37             for (int j = 0; j < n; ++j)
38                 adj[i][j] += adj[i][k] * adj[k][j];
39 }
40
41 bool isNegativeCycle(int n, const vector<vector<int>>& adj) {
42     // run floyd first
43     for (int i = 0; i < n; ++i)
44         if (adj[i][i] < 0)
45             return true;
46     return false;
47 }
48
49 bool anyEffectiveCycle(int n, const vector<vector<int>>& adj,
50     int src, int dest, int 00) {
51     // run floyd first
52     for (int i = 0; i < n; ++i)
53         if (adj[i][i] < 0 && adj[src][i] < 00 && adj[i][dest] < 00)
54             return true;
55     return false;
56 }

```

Topological Sort

```

1  queue<int> queue;
2  for (int i = 0; i < n; i++)
3      if (in_degree[i] == 0) { queue.push(i); }
4
5  vector<int> top_sort;
6  while (!queue.empty()) {
7      int curr = queue.front();
8      queue.pop();
9      top_sort.push_back(curr);
10     for (int next : graph[curr]) {

```

```

11         if (--in_degree[next] == 0) { queue.push(next); }
12     }
13 }

```

Bipartite Check

```

1  int n;
2  vector<vector<int>> adj;
3
4  vector<int> side(n, -1);
5  bool is_bipartite = true;
6  queue<int> q;
7  for (int st = 0; st < n; ++st) {
8      if (side[st] == -1) {
9          q.push(st);
10         side[st] = 0;
11         while (!q.empty()) {
12             int v = q.front();
13             q.pop();
14             for (int u : adj[v]) {
15                 if (side[u] == -1) {
16                     side[u] = side[v] ^ 1;
17                     q.push(u);
18                 } else {
19                     is_bipartite &= side[u] != side[v];
20                 }
21             }
22         }
23     }
24 }

```

Tree Diameter

```

1  pair<int, int> dfs(int u, int p) {
2      pair<int, int> ret = {0, u};
3      for (int v : tree[u]) {
4          if (v == p) continue;
5          auto x = dfs(v, u);
6          ret = max(ret, {x.first + 1, x.second});
7      }
8      return ret;
9  }

```

Tree

```

1  struct tree {
2      int root = 0;
3      vector<vector<int>> g;
4      explicit tree(int n) : g(n) { }
5      void add(int u, int v) {
6          g[u].push_back(v);
7          g[v].push_back(u);
8      }
9
10     vector<int> &operator[](int u) {

```

```

11     return g[u];
12 }
13
14 int cntDfs = 0;
15 vector<int> in, out, lvl, sz, top, par, seq;
16
17 void init(int rt = 0) {
18     root = rt;
19     in = out = lvl = top = par = seq = vector<int>(g.size());
20     sz.resize(g.size(), 1);
21     par[root] = top[root] = root;
22     dfs(root);
23     dfs2(root);
24 }
25
26 void dfs(int u) {
27     for(int &v : g[u]) {
28         lvl[v] = lvl[u] + 1;
29         par[v] = u;
30         g[v].erase(find(g[v].begin(), g[v].end(), u));
31         dfs(v);
32         sz[u] += sz[v];
33         if(sz[v] > sz[g[u][0]])
34             swap(v, g[u][0]);
35     }
36 }
37 void dfs2(int u) {
38     in[u] = cntDfs++;
39     seq[in[u]] = u;
40     for(int v : g[u]) {
41         top[v] = v == g[u][0]? top[u]: v;
42         dfs2(v);
43     }
44     out[u] = cntDfs - 1;
45 }
46
47 int jump(int u, int k) {
48     if(k > lvl[u]) return -1;
49
50     int d = lvl[u] - k;
51     while (lvl[top[u]] > d) {
52         u = par[top[u]];
53     }
54
55     return seq[in[u] - lvl[u] + d];
56 }
57
58 bool isAncestor(int u, int v) {
59     return in[u] <= in[v] && in[v] <= out[u];
60 }
61
62 int lca(int u, int v) {
63     if(lvl[u] > lvl[v]) swap(u, v);
64     if(isAncestor(u, v)) return u;
65     while (top[u] != top[v]) {
66         if (lvl[top[u]] > lvl[top[v]]) {
67             u = par[top[u]];

```

```

68         } else {
69             v = par[top[v]];
70         }
71     }
72     return lvl[u] < lvl[v]? u: v;
73 }
74
75 int dis(int u, int v) {
76     return lvl[u] + lvl[v] - 2 * lvl[lca(u, v)];
77 }
78 };

```

Binary Lifting

```

1  int Log = __lg(n) + 1;
2  vector<vector<int>> lift(n+1, vector<int>(Log, -1));
3  function<void(int, int)> dfs2 = [&](int u, int p) {
4      for (auto v : tree[u]) if (v != p) {
5          lift[v][0] = u;
6          for (int l = 1; l < Log && ~lift[v][l-1]; l++)
7              lift[v][l] = lift[ lift[v][l-1] ][l-1];
8          dfs2(v, u);
9      }
10 };
11 dfs2(root, -1);
12
13 if (depth[u] < depth[v])
14     swap(u, v);
15 int k = depth[u] - depth[v];
16
17 for (int l = Log-1; ~l && ~u; l--) {
18     if (k >> l & 1)
19         u = lift[u][l];
20 }
21 if (u == v) return u;
22 for (int l = Log-1; l > -1; l--) {
23     if (lift[u][l] != lift[v][l])
24         u = lift[u][l], v = lift[v][l];
25 }
26 return lift[u][0];

```

LCA

```

1  struct LCA {
2      int n, m, LOG;
3      vector<vector<int>> g;
4      vector<int> euler, first, depth, lg;
5      vector<vector<int>> st;
6
7      LCA(const vector<vector<int>>& tree, int root) {
8          n = tree.size();
9          g = tree;
10         first.assign(n, -1);
11         depth.assign(n, 0);
12
13         dfs(root, -1, 0);

```



```

14     m = euler.size();
15
16     lg.assign(m+1, 0);
17     for (int i = 2; i <= m; i++)
18         lg[i] = lg[i>>1] + 1;
19     LOG = lg[m] + 1;
20
21     st.assign(m, vector<int>(LOG));
22     for (int i = 0; i < m; i++)
23         st[i][0] = euler[i];
24
25     for (int j = 1; j < LOG; j++) {
26         for (int i = 0; i + (1<<j) <= m; i++) {
27             int x = st[i][j-1], y = st[i + (1<<(j-1))][j-1];
28             st[i][j] = (depth[x] < depth[y] ? x : y);
29         }
30     }
31 }
32
33 void dfs(int u, int p, int d) {
34     if (first[u] == -1) first[u] = euler.size();
35     euler.push_back(u);
36     depth[u] = d;
37     for (int v : g[u]) {
38         if (v == p) continue;
39         dfs(v, u, d+1);
40         euler.push_back(u);
41     }
42 }
43
44 int lca(int u, int v) {
45     int L = first[u], R = first[v];
46     if (L > R) swap(L, R);
47     int len = R - L + 1, k = lg[len];
48     int x = st[L][k], y = st[R - (1<<k) + 1][k];
49     return depth[x] < depth[y] ? x : y;
50 }
51
52 int dis(int u, int v) {
53     return depth[u] + depth[v] - 2 * depth[lca(u, v)];
54 }
55 };

```

DSU on Trees

```

1 void pre(int u) {
2     sz[u] = 1;
3     for (int &v : tree[u]) {
4         tree[v].erase(find(tree[v].begin(), tree[v].end(), u));
5         pre(v);
6         sz[u] += sz[v];
7         if (sz[v] > sz[tree[u].front()]) swap(v, tree[u].front());
8     }
9 }
10
11 void addver(int u) {

```

```

12 }
13 }
14 void removever(int u) {
15
16 }
17
18 void addsub(int u) {
19     addver(u);
20     for (int v : tree[u]) addsub(v);
21 }
22 void removesub(int u) {
23     removever(u);
24     for (int v : tree[u]) removesub(v);
25 }
26
27 void dfs(int u) {
28     for (int v : tree[u]) if (v != tree[u].front()) {
29         dfs(v);
30         removesub(v);
31     }
32     if (!tree[u].empty())
33         dfs(tree[u].front());
34     addver(u);
35     for (int v : tree[u])
36         if (v != tree[u].front())
37             addsub(v);
38
39     // query[u]
40 }

```

SCC / Strongly Connected Componenets

```

1  // --- USAGE REQUIREMENTS ---
2  // 1. timer must be declared and initialized to 0 outside the function.
3  // 2. g is your 0-based directed adjacency list (vector<vector<int>>).
4  // 3. v is a 0-based array/vector of vertex weights
5     (used to find the min weight per SCC).
6  // 4. idsccl[i] will hold the 0-based SCC ID for vertex i.
7  // 5. cond generates the condensed DAG where each node is a distinct SCC.
8
9  // --- MODIFICATIONS FOR BRIDGES & ARTICULATION POINTS (UNDIRECTED GRAPH) ---
10 // 1. Update the lambda signature to pass the parent: tarj = [&](int u, int p = -1)
11 // 2. Add int children = 0; at the top of the lambda to
12     count root children for APs.
13 // 3. Ignore the back-edge to the parent in the loop: if (v_ == p) continue;
14 // 4. You can completely remove vis, stk, mn, and idsccl
15     arrays if you are not extracting SCCs.
16
17 vector<int> tin(n, -1), low(n), vis(n), stk, mn, idsccl(n, -1);
18 function<void(int)> tarj = [&](int u) { // Change to: (int u, int p = -1)
19     tin[u] = low[u] = timer++;
20     vis[u] = 1; // Marks u as currently in the active SCC stack
21     stk.push_back(u);
22
23     // int children = 0; // Uncomment for Articulation Points
24

```

```

25     for (int v_ : g[u]) {
26         // Uncomment to prevent traversing back to parent in undirected graphs
27         // if (v_ == p) continue;
28
29         if (tin[v_] == -1) {
30             // children++; // Uncomment for Articulation Points
31             tarj(v_); // Change to: tarj(v_, u)
32             low[u] = min(low[u], low[v_]);
33
34             // --- BRIDGE CHECK ---
35             // if (low[v_] > tin[u]) { /* Edge (u, v_) is a bridge */ }
36
37             // --- ARTICULATION POINT CHECK (NON-ROOT) ---
38             // if (low[v_] >= tin[u] && p != -1) { /* Vertex u is an AP */ }
39
40             } else if (vis[v_]) {
41                 // Back-edge found. v_ is already visited AND
42                 // still in the current stack
43                 low[u] = min(low[u], tin[v_]);
44             }
45     }
46
47     // --- SCC EXTRACTION LOGIC ---
48     // If u is the root of an SCC, pop all vertices belonging to
49     // it from the stack
50     if (low[u] == tin[u]) {
51         int t;
52         mn.push_back(1e5); // Initialize the minimum weight for this new SCC
53         do {
54             t = stk.back();
55             stk.pop_back();
56             vis[t] = 0; // Mark as removed from the active stack
57             // Track min value (requires your v array)
58             mn.back() = min(mn.back(), v[t]);
59             // Assign the new SCC ID to vertex t
60             idsccl[t] = (int)mn.size() - 1;
61         } while (t ^ u);
62     }
63
64     // --- ARTICULATION POINT CHECK (ROOT) ---
65     // if (p == -1 && children > 1) { /* Vertex u (the root) is an AP */ }
66 };
67
68 for (int i = 0; i < n; i++)
69     if (tin[i] == -1)
70         tarj(i); // Change to: tarj(i, -1)
71
72 // --- BUILD THE CONDENSED DAG ---
73 vector<vector<int>> cond(mn.size());
74 for (int i = 0; i < n; i++)
75     for (int u : g[i])
76         if (idsccl[i] ^ idsccl[u]) // If an edge connects two different SCCs
77             cond[idsccl[i]].push_back(idsccl[u]);

```

```

1  template<class T>
2  struct WLCA {
3      int n, Log;
4      vector<vector<int>>> up;
5      vector<vector<T>>> val;
6      vector<int> lvl;
7      explicit WLCA(vector<vector<pair<int, T>>> &g, int root = 0) : n((int)g.size()),
8          lvl(n), Log(__lg(n) + 1), up(n, vector<int>(Log, root)),
9          val(n, vector<T>(Log)) {
10         function<void(int)> dfs = [&](int u) -> void {
11             for(auto [v, w] : g[u]) {
12                 if(v == up[u][0]) continue;
13                 lvl[v] = lvl[u] + 1, up[v][0] = u, val[v][0] = w;
14                 for(int l = 1; l < Log; l++) {
15                     up[v][l] = up[up[v][l - 1]][l - 1];
16                     val[v][l] = val[v][l - 1] + val[up[v][l - 1]][l - 1];
17                 }
18                 dfs(v);
19             }
20         };
21         dfs(root);
22     }
23     pair<int, T> k_ancestor(int u, int k) {
24         T ans;
25         while(k) {
26             ans = ans + val[u][__builtin_ctz(k)];
27             u = up[u][__builtin_ctz(k)];
28             k &= k - 1;
29         }
30         return {u, ans};
31     }
32     int lca(int u, int v) {
33         if(lvl[u] < lvl[v]) swap(u, v);
34         u = k_ancestor(u, lvl[u] - lvl[v]).first;
35         if(u == v) return u;
36         for(int l = Log - 1; l >= 0; l--) {
37             if(up[u][l] ^ up[v][l]) {
38                 u = up[u][l], v = up[v][l];
39             }
40         }
41         return up[u][0];
42     }
43     int dis(int u, int v, int l = -1) {
44         if(l == -1) l = lca(u, v);
45         return lvl[u] + lvl[v] - 2 * lvl[l];
46     }
47 };

```

Rerooter

```

1  namespace reroot {
2      const auto exclusive = [](const auto& a, const auto& base,
3          const auto& merge_into, int vertex) {
4          int n = (int)a.size();
5          using Aggregate = decay_t<decltype(base)>;
6          vector<Aggregate> b(n, base);

```

```

7      for (int bit = __lg(n); bit >= 0; --bit) {
8          for (int i = n - 1; i >= 0; --i) b[i] = b[i >> 1];
9          int sz = n - (n & !bit);
10         for (int i = 0; i < sz; ++i) {
11             int index = (i >> bit) ^ 1;
12             b[index] = merge_into(b[index], a[i], vertex, i);
13         }
14     }
15     return b;
16 };
17 // MergeInto : Aggregate * Value * Vertex(int) * EdgeIndex(int) -> Aggregate
18 // Base : Vertex(int) -> Aggregate
19 // FinalizeMerge : Aggregate * Vertex(int) * EdgeIndex(int) -> Value
20 const auto rerooter = [](const auto& g, const auto& base,
21     const auto& merge_into, const auto& finalize_merge) {
22     int n = (int)g.size();
23     using Aggregate = decay_t<decltype(base(0))>;
24     using Value = decay_t<decltype(finalize_merge(base(0), 0, 0))>;
25     vector<Value> root_dp(n), dp(n);
26     vector<vector<Value>> edge_dp(n), redge_dp(n);
27
28     vector<int> bfs, parent(n);
29     bfs.reserve(n);
30     bfs.push_back(0);
31     for (int i = 0; i < n; ++i) {
32         int u = bfs[i];
33         for (auto v : g[u]) {
34             if (parent[u] == v) continue;
35             parent[v] = u;
36             bfs.push_back(v);
37         }
38     }
39
40     for (int i = n - 1; i >= 0; --i) {
41         int u = bfs[i];
42         int p_edge_index = -1;
43         Aggregate aggregate = base(u);
44         for (int edge_index = 0; edge_index < (int)g[u].size(); ++edge_index) {
45             int v = g[u][edge_index];
46             if (parent[u] == v) {
47                 p_edge_index = edge_index;
48                 continue;
49             }
50             aggregate = merge_into(aggregate, dp[v], u, edge_index);
51         }
52         dp[u] = finalize_merge(aggregate, u, p_edge_index);
53     }
54
55     for (auto u : bfs) {
56         dp[parent[u]] = dp[u];
57         edge_dp[u].reserve(g[u].size());
58         for (auto v : g[u]) edge_dp[u].push_back(dp[v]);
59         auto dp_exclusive = exclusive(edge_dp[u], base(u), merge_into, u);
60         redge_dp[u].reserve(g[u].size());
61         for (int i = 0; i < (int)dp_exclusive.size(); ++i)
62             redge_dp[u].push_back(finalize_merge(dp_exclusive[i], u, i));

```

```

63         root_dp[u] = finalize_merge(n > 1 ?
64             merge_into(dp_exclusive[0], edge_dp[u][0], u, 0) :
65             base(u), u, -1);
66         for (int i = 0; i < (int)g[u].size(); ++i) {
67             dp[g[u][i]] = redge_dp[u][i];
68         }
69     }
70
71     return make_tuple(move(root_dp), move(edge_dp), move(redge_dp));
72 };
73 } // namespace reroot
74 // [&](Aggregate agg, Aggregate chdp, int v, int eid)
75 // [&](Aggregate agg, int v, int eid);

```

Dynamic Connectivity

```

1  struct Query {
2      char t;
3      int u, v;
4  };
5
6  struct Elem {
7      int u, v, szU, cnt;
8  };
9
10 struct DSURollback {
11     // offline : [+ a b] add edge between a, b
12     //           [- a b] remove edge between a, b
13     //           [?] number of connected components
14     int cnt;
15     stack<Elem> st;
16     vector<bool> ans;
17     vector<int> sz, par;
18     map<int, vector<pair<int, int>>> g;
19
20     DSURollback(int n) {
21         cnt = n;
22         par.resize(n + 1);
23         sz.resize(n + 1, 1);
24         iota(par.begin(), par.end(), 0);
25     }
26
27     void rollback(int x) {
28         while (st.size() > x) {
29             auto e = st.top();
30             st.pop();
31             cnt = e.cnt;
32             sz[e.u] = e.szU;
33             par[e.v] = e.v;
34         }
35     }
36
37     int findSet(int u) {
38         return par[u] == u ? u : findSet(par[u]);
39     }
40
41     void update(int u, int v) {

```

```

42     st.push({u, v, sz[u], cnt});
43     cnt--;
44     par[v] = u;
45     sz[u] += sz[v];
46 }
47
48 void unionSet(int u, int v) {
49     u = findSet(u);
50     v = findSet(v);
51     if (u != v) {
52         if (sz[u] < sz[v])
53             swap(u, v);
54         update(u, v);
55     }
56 }
57
58 void solve(int x, int l, int r) {
59     int cur = st.size();
60
61     for (auto i: g[x])
62         unionSet(i.first, i.second);
63
64     if (l == r) {
65         if (ans[l])
66             cout << cnt << endl;
67         rollback(cur);
68         return;
69     }
70     int m = (l + r) >> 1;
71     solve(x * 2, l, m);
72     solve(x * 2 + 1, m + 1, r);
73     rollback(cur);
74 }
75
76 void traverse(int x, int lX, int rX, int l, int r, int u, int v) {
77     if (rX < l || lX > r)
78         return;
79     if (lX >= l && rX <= r) {
80         g[x].emplace_back(u, v);
81         return;
82     }
83     int m = (lX + rX) >> 1;
84     traverse(x * 2, lX, m, l, r, u, v);
85     traverse(x * 2 + 1, m + 1, rX, l, r, u, v);
86 }
87
88 void build(vector<Query> &queries) {
89     int q = queries.size();
90     ans.resize(q);
91     map<pair<int, int>, vector<pair<int, int>>> mp;
92     for (int i = 0; i < queries.size(); i++) {
93         auto cur = queries[i];
94         if (cur.u > cur.v)
95             swap(cur.u, cur.v);
96         if (cur.t == '?')
97             ans[i] = 1;

```

```

98         else if (cur.t == '+')
99             mp[{cur.u, cur.v}].emplace_back(i, queries.size());
100         else {
101             mp[{cur.u, cur.v}].back().second = i - 1;
102             traverse(1, 0, q - 1, mp[{cur.u, cur.v}].back().first,
103                 mp[{cur.u, cur.v}].back().second, cur.u, cur.v);
104         }
105     }
106
107     for (auto i: mp) {
108         for (auto j: i.second) {
109             if (j.second == q)
110                 traverse(1, 0, q - 1, j.first, q - 1, i.first.first,
111                     i.first.second);
112         }
113     }
114 }
115 };
116
117 void testCase() {
118     int n, q;
119     cin >> n >> q;
120     if (!q)
121         return;
122     DSURollback dsu(n);
123     vector<Query> queries(q);
124     char t;
125     int x, y;
126     for (int i = 0; i < q; i++) {
127         cin >> t;
128         if (t == '?')
129             queries[i] = {t, 0, 0};
130         else {
131             cin >> x >> y;
132             if (y < x)
133                 swap(x, y);
134             queries[i] = {t, x, y};
135         }
136     }
137
138     dsu.build(queries);
139     dsu.solve(1, 0, q - 1);
140 }

```

Max Flow (Dinic)

```

1  class Dinic { //  $O(n^2 * m)$ , 0-based
2      private:
3          struct E {
4              int to, rev;
5              ll c, oc;
6              ll f() { return max(oc - c, 0LL); }
7          };
8          vector<int> lvl, ptr, q;
9          vector<vector<E>> g;
10         ll dfs(int v, int t, ll f) {

```



```

11         if (v == t || !f) return f;
12         for (int &i = ptr[v]; i < g[v].size(); ++i) {
13             E &e = g[v][i];
14             if (lvl[e.to] == lvl[v] + 1 && e.c)
15                 if (ll p = dfs(e.to, t, min(f, e.c))) {
16                     e.c -= p;
17                     g[e.to][e.rev].c += p;
18                     return p;
19                 }
20         }
21         return 0;
22     }
23
24 public:
25     Dinic(int n) : lvl(n), ptr(n), q(n), g(n) {}
26     void add_edge(int u, int v, ll c) { // directed
27         g[u].push_back({v, (int)g[v].size(), c, c});
28         g[v].push_back({u, (int)g[u].size() - 1, 0, 0});
29     }
30     ll calc(int s, int t) { // source to destination
31         ll flow = 0;
32         while (true) {
33             fill(lvl.begin(), lvl.end(), 0);
34             int qs = 0, qe = 0;
35             lvl[q[qs++]] = s;
36             while (qs < qe && !lvl[t]) {
37                 int v = q[qs++];
38                 for (auto &e : g[v])
39                     if (!lvl[e.to] && e.c) lvl[q[qs++]] = e.to = lvl[v] + 1;
40             }
41             if (!lvl[t]) break;
42             fill(ptr.begin(), ptr.end(), 0);
43             while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
44         }
45         return flow;
46     }
47     void reset() { // before calling calc again
48         for (auto &adj : g)
49             for (auto &e : adj) e.c = e.oc;
50     }
51     bool leftOfMinCut(int a) { return lvl[a] != 0; }
52 };

```

Max Bipartite Matching (Karp) [with building]

```

1 struct matching {
2     int nl, nr;
3     vector<vector<int>>> g;
4     vector<int> dis, ml, mr;
5     explicit matching(int nl, int nr) : nl(nl), nr(nr), g(nl), dis(nl), ml(nl, -1),
6         mr(nr, -1) {}
7
8     void add(int l, int r) { // [i, j]
9         g[l].push_back(r);
10    }
11

```

```

12 void bfs() {
13     queue<int> q;
14     for(int u = 0; u < nL; u++) {
15         if(mL[u] == -1) q.push(u), dis[u] = 0;
16         else dis[u] = -1;
17     }
18     while(!q.empty()) {
19         int l = q.front(); q.pop();
20         for(int r : g[l]) {
21             if(mr[r] != -1 && dis[mr[r]] == -1)
22                 q.push(mr[r]), dis[mr[r]] = dis[l] + 1;
23         }
24     }
25 }
26
27 bool canMatch(int l) {
28     for(int r : g[l]) if(mr[r] == -1)
29         return mr[r] = l, mL[l] = r, true;
30     for(int r : g[l]) if(dis[l] + 1 == dis[mr[r]] && canMatch(mr[r]))
31         return mr[r] = l, mL[l] = r, true;
32     return false;
33 }
34
35 int maxMatch() {
36     int ans = 0, turn = 1;
37     while(turn) {
38         bfs(), turn = 0;
39         for(int l = 0; l < nL; l++) if(mL[l] == -1)
40             turn += canMatch(l);
41         ans += turn;
42     }
43     return ans;
44 }
45
46 pair<vector<int>, vector<int>> minCover() {
47     vector<int> L, R;
48     for (int u = 0; u < nL; ++u) {
49         if(dis[u] == -1) L.push_back(u);
50         else if(mL[u] != -1) R.push_back(mL[u]);
51     }
52     return {L, R};
53 }
54 };

```

4. Math and Number Theory

Math

```
1 Stirling numbers of the first kind : the number of permutations of n elements with k disjoint
2  $S(n,k) = n * S(n-1,k) + S(n-1,k-1)$ 
3 Sum  $k = 0 \rightarrow n$  of  $S(n,k) = n!$ 
4
5 Stirling numbers of the second kind : the number of ways to partition
6 a set of n elements into k nonempty subsets
7  $S(n,k) = k * S(n-1,k) + S(n-1,k-1)$ 
8 Sum  $k = 0 \rightarrow n$  of  $S(n,k) = B_n$ 
9
10 Bell numbers : the possible partitions of a set into nonempty subsets
11 Sum  $k = 0 \rightarrow n-1$  of  $n-1Ck * B_k = B_n$ 
12
13 Sum of first n even numbers :  $n*(n+1)$ 
14 Sum of first n odd numbers :  $n*n$ 
15
16 Sum of squares of first n numbers:
17  $n*(n+1)*(2*n+1)/6$ 
18 Sum of squares of first n even numbers:
19  $2*n*(n+1)*(2*n+1)/3$ 
20 Sum of squares of first n odd numbers:
21  $n*(2*n+1)*(2*n-1)/3$ 
22
23 Number of ways to pick equal number of elements from two sets :  $(n+m)C(m)$ 
24
25 Sum of  $\phi(d)$  for all  $d | n$  is equal to n.
26 Number of pairs (x,y) that satisfy  $x+y=n$  and  $\gcd(x,y)=1$  is  $\phi(n)$ .
27
28 Sum  $(nCk)$  for  $k [0,n] = 2^n$ 
29 Sum  $(mCk)$  for all  $m [0,n] = (n+1)C(k+1)$ 
30 Sum  $((n+k)Ck)$  for all  $k [0,m] = (n+m+1)C(m)$ 
31 Sum  $((nCk)^2)$  for all  $k [0,n] = (2*n)Cn$ 
32 Sum  $(i*nCi)$  for all  $i [1,n] = n*2^{(n-1)}$ 
33 Sum  $((n-i)Ci)$  for all  $i [0,n] = F(n+1)$ 
34 Number of arrays with size n and sum m =  $(n-1+m)C(m) = (n-1+m)C(n-1)$ 
35
36  $P(A|B)$  is the probability of event A given that event B happened.
37  $P(A\&B)$  is the probability of events A and B happening.
38
39  $P(A|B) = (P(B|A) * P(A)) / P(B)$ 
40  $P(A|B) = P(A\&B) / P(B)$ 
41
42 Divisibility:
43 2 if the rightmost digit is divisible by 2
44 3 if the sum of the digits is divisible by 3
45 4 if the number formed by the last two digits is divisible by 4
46 5 if the rightmost digit is 0 or 5
47 6 if it is divisible by 2 and 3
48 7 if The number formed by all digits except the right-most digit -
49 (2 * right-most digit) is divisible by 7
50 8 if the number formed by the last 3 digits is divisible by 8
51 9 if the sum of the digits is divisible by 9
52
53 -Getting half of the binomial expansion (Only odd indices or only even indices)
```

```

54     by using  $(a+b)^n$  and  $(a-b)^n$  and adding both of them
55
56 To solve quadratic equation  $ax^2 + bx + c = 0$ 
57  $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ 
58
59 -Sum of geometric series  $ar^i$  (i from 0 to n) =  $a(1 - r^{n+1}) / (1 - r)$ 
60 -Sum of geometric series  $ar^i$  (i from 0 to infinity) =  $a / (1 - r)$ 
61
62 - XOR from 1 to n =  $n \% 4$  [n, 1, n+1, 0]
63
64 - n can be sum of 2 squares if n has no  $p^k$  [p = 3 % 4, k is odd]
65     in its prime factorization
66 - n can be sum of 3 squares if n is not in form  $4^a(8b + 7)$ 
67 - n always can be sum of  $\geq 4$  squares (remaining are zeros)
68
69 ===== FADY THE GOOOOAAAT =====
70     sum of divisors
71      $\text{prime}^{\text{power}} * \text{prime}_2^{\text{power}_2} * \dots$ 
72
73      $((\text{prime}^{(\text{power} + 1)} - 1) / (\text{prime} - 1)) * ((\text{prime}_2^{(\text{power}_2 + 1)} - 1) / (\text{prime}_2 - 1)) * \dots$ 
74 =====
75     a % m == b
76     a and m not coprime
77     g = gcd(a, m)
78      $(a / g) \% (m / g) = b / g$ 
79
80      $a^x \% m == b$ 
81     a and m not coprime
82     g = gcd(a, m)
83      $(a^{(x-1)} * (a / g)) \% (m / g) = b / g$ 
84 =====
85      $a^{(\text{power} \% \phi(m))} \% m;$ 
86 =====
87     count balanced brackets
88      $r = n/2$  || or r = number of opened brackets
89      $nCr(n, r) - nCr(n, r-1)$ 
90 =====
91     // different n*n grids whose each square have m colors
92     // if possible to rotate one of them so that they look the same then they same
93     t = n * n;
94     total = (fp(m, t)
95             + fp(m, (t + 1) / 2)
96             + 2 * fp(m, (t / 4) + (n % 2))) % mod;
97     total = mul(total, fp(4, mod - 2));
98 =====
99     biggest divisors
100    735134400 1344 => 2^6 3^3 5^2 7 11 13 17
101    73513440 768
102 =====
103    for (int x = mask; x > 0; x = (x - 1) & mask)
104        get all x such that mask = mask | x
105 =====
106    sum from 1 to n:  $i * nCr(n, i) = n * (1LL \ll (n - 1))$ 
107    sum of odd between [1, n] =  $((n+1) / 2)^2$ 
108    sum of even between [1, n] =  $(n/2) * (n/2 + 1)$ 
109
110 ll sum_total(ll l, ll r) { // sum [l, r]

```

```

111     return (r - l + 1) * (l + r) / 2;
112 }
113
114 ll sum_even(ll l, ll r) { // sum even [l, r]
115     if (l % 2 != 0) ++l;
116     if (r % 2 != 0) --r;
117     if (l > r) return 0;
118     ll n = (r - l) / 2 + 1;
119     return n * (l + r) / 2;
120 }
121
122 ll sum_odd(ll l, ll r) { // sum odd [l, r]
123     if (l % 2 == 0) ++l;
124     if (r % 2 == 0) --r;
125     if (l > r) return 0;
126     ll n = (r - l) / 2 + 1;
127     return n * (l + r) / 2;
128 }
129
130 ll arithm1(ll l, ll r, ll a, ll d) { // [l, r] starting a and diff d
131     if (d == 0) return (a >= l && a <= r) ? a : 0;
132     ll n1 = ((l - a + d - (d > 0 ? 1 : -1)) / d);
133     ll first = a + n1 * d;
134     if ((d > 0 && first > r)
135         || (d < 0 && first < r))
136         return 0;
137     ll n2 = ((r - a) / d);
138     ll last = a + n2 * d;
139     ll n = (n2 - n1 + 1);
140     return n * (first + last) / 2;
141 }
142
143 ll arithm2(ll a, ll d, ll n) { // starting a, diff d, n terms
144     return n * (2 * a + (n - 1) * d) / 2;
145 }
146 */
147
148 ll phi(ll x) { // sqrt(x)
149     ll ans = x;
150     for(ll i = 2; i * i <= x; i++) {
151         if(x % i == 0) {
152             while(x % i == 0) x /= i;
153             ans -= ans / i;
154         }
155     }
156     if(x > 1) ans -= ans / x;
157     return ans;
158 }
159
160 array<ll, 2> CRT(ll a1, ll m1, ll a2, ll m2) {
161     // x = a1 % m1, x = a2 % m2
162     a1 %= m1, a2 %= m2;
163     auto [g, q1, q2] = eGcd(m1, -m2);
164     if ((a2 - a1) % g) return {-1, -1};
165     ll lcm = m1 / g * m2;
166     ll m = m2 / g;

```

```

167     q1 = (a2 - a1) / g % m * q1 % m;
168     ll res = (a1 + m1 * q1) % lcm;
169     if (res < 0) res += lcm;
170     return {res, lcm};
171 }
172
173 ll BSGS(ll a, ll b, ll p) { //  $a^x = b \pmod p$ 
174     a %= p, b %= p;
175     if(b == 1) return 0;
176     if(a == 0) return b == 0? 1: -1;
177     int add = 0;
178     ll g, tmp = 1;
179     while ((g = gcd(a, p)) > 1) {
180         if(b % g) return -1;
181         p /= g, b /= g, tmp = tmp * (a / g) % p, ++add;
182         if(tmp == b) return add;
183     }
184     b = b * modInv(tmp, p) % p;
185     int n = (int)sqrtl(p) + 1;
186     unordered_map<ll, int> mp;
187     for (ll q = 0, cur = 1; q <= n; ++q)
188         mp.emplace(cur, q), cur = cur * a % p;
189     ll an = 1;
190     for (ll i = 0; i < n; ++i) an = an * a % p;
191     an = modInv(an, p);
192     for (ll i = 0, cur = b; i <= n; ++i) {
193         auto it = mp.find(cur);
194         if(it != mp.end()) return i * n + it->second + add;
195         cur = cur * an % p;
196     }
197     return -1;
198 }
199
200 int sumNPowerM(int n, int m) { //  $1^m + 2^m \dots n^m$ 
201     int k = m + 3;
202     vector<int> res(k);
203     for(int i = 1; i < k; i++) res[i] = (res[i - 1] + fp(i, m)) % mod;
204     if(n < k) return res[n];
205     int facK = k;
206     vector<int> p(k); p[0] = 1;
207     for(int i = 1; i < k; i++) {
208         p[i] = int(p[i - 1] * 1LL * (n - i) % mod);
209         facK = int(facK * 1LL * i % mod);
210     }
211     vector<int> inv(k + 1), s(k + 1);
212     inv[k] = fp(facK, mod - 2), s[k] = 1;
213     for(int i = k - 1; i >= 0; i--) {
214         s[i] = int(s[i + 1] * 1LL * (n - i) % mod);
215         inv[i] = int(inv[i + 1] * 1LL * (i + 1) % mod);
216     }
217     int ans = 0;
218     for(int i = 1; i < k; i++) {
219         int cur = int(res[i] * 1LL * p[i - 1] % mod * s[i + 1] % mod *
220             inv[i - 1] % mod * inv[k - i - 1] % mod);
221         if((k - i + 1) & 1) cur = (mod - cur) % mod;
222         ans = (ans + cur) % mod;

```

```

223     }
224     return ans;
225 }

```

ModInt

```

1  template <int M> struct ModInt {
2      template <class T> T binpow(T a, ll b) {
3          T res = 1;
4          while (b) { if (b & 1) res *= a; a *= a, b >>= 1; }
5          return res;
6      }
7      int v;
8      ModInt() : v(0) {}
9      ModInt(ll v_) {
10         v = v_ % M;
11         if (v < 0) v += M;
12     }
13
14     bool operator==(ModInt o) const { return v == o.v; };
15     bool operator!=(ModInt o) const { return !(*this == o); };
16
17     // ModInt add(ModInt a, ModInt b) { return ((a.v + b.v % M)+M)%M; }
18     // ModInt sub(ModInt a, ModInt b) { return add(a, -b); }
19     // ModInt mul(ModInt a, ModInt b) { return 1ll * a.v * b.v % M; }
20     // ModInt div(ModInt a, ModInt b) { return mul(a, binpow(b, M-2)); }
21
22     ModInt &operator+=(ModInt o) { v = (v + o.v) % M; return *this; }
23     ModInt &operator-=(ModInt o) { v = (v - o.v + M) % M; return *this; }
24     ModInt &operator*=(ModInt o) { v = 1ll * v * o.v % M; return *this; }
25     ModInt &operator/=(ModInt o) { return (*this *= binpow(o, M - 2)); }
26
27     friend ModInt operator+(ModInt a, ModInt b) { return a += b; }
28     friend ModInt operator-(ModInt a, ModInt b) { return a -= b; }
29     friend ModInt operator*(ModInt a, ModInt b) { return a *= b; }
30     friend ModInt operator/(ModInt a, ModInt b) { return a /= b; }
31     ModInt operator-() { return 0 - *this; }
32
33     friend istream &operator>>(istream &is, ModInt &a) {
34         ll x; is >> x;
35         a = ModInt(x);
36         return is;
37     }
38     friend ostream &operator<<(ostream &os, ModInt a) { return os << a.v; }
39     friend string to_string(ModInt a) { return to_string(a.v); }
40 };
41 using mint = ModInt<>;

```

Sieve / PHI up to n / 2D gcd()

```

1  const int NS = 1e7;
2  const int NP = (NS/log(NS)) * (1 + 1.28 / log(NS));
3  int prSz;
4  int spf[NS], prm[NP];
5

```

```

6  auto pre_Sieve = []() {
7      for (int i = 2; i < NS; i++){
8          if(!spf[i]) spf[i] = prm[prSz++] = i;
9          for(int j = 0; i * prm[j] < NS; j++) {
10             spf[i * prm[j]] = prm[j];
11             if(spf[i] == prm[j]) break;
12         }
13     }
14     return 0;
15 }();
16
17 auto factors(int n) {
18     vector<array<int, 2>> res;
19     if(n < 2) return res;
20     int p = spf[n];
21     while(p > 1) {
22         res.push_back({p, 0});
23         while(n % p == 0) n /= p, res.back()[1]++;
24         p = spf[n];
25     }
26     return res;
27 }
28
29 auto getDivisors(int _n) {
30     auto _fac = factors(_n);
31     int cnt = 1;
32     for(auto [pr, pw] : _fac) cnt *= pw + 1;
33     vector<int> res(1, 1); res.reserve(cnt);
34
35     for(auto [pr, pw] : _fac)
36         for(int i = int(res.size()) - 1; i >= 0; i--)
37             for(int b = pr, j = 0; j < pw; j++, b *= pr)
38                 res.push_back(res[i] * b);
39     sort(res.begin(), res.end());
40     return res;
41 }
42
43 bool isPrime(ll n) {
44     if(n < 4) return n > 1;
45     if(n % 2 == 0 || n % 3 == 0) return false;
46     for (ll i = 5; i * i <= n; i += 6)
47         if (n % i == 0 || n % (i + 2) == 0)
48             return false;
49     return true;
50 }
51
52 void phi_1_to_n(int n) {
53     vector<int> phi(n + 1);
54     for (int i = 0; i <= n; i++)
55         phi[i] = i;
56
57     for (int i = 2; i <= n; i++) {
58         if (phi[i] == i) {
59             for (int j = i; j <= n; j += i)
60                 phi[j] -= phi[j] / i;
61         }

```



```

62     }
63 }
64
65 // 2d gcd in n^2
66 vector<vector<int>> GCD(N, vector<int>(N, 1));
67 for(int d = 2; d < N; ++d)
68     for(int i = d; i < N; i += d)
69         for(int j = d; j < N; j += d)
70             GC[i][j] = d;

```

Sieve up to 1e9

```

1  vector<int> sieve(const int BN, const int Q = 17, const int L = 1 << 15) {
2      using u = uint32_t; using u8 = uint8_t;
3      static const u rs[] = {1, 7, 11, 13, 17, 19, 23, 29};
4      struct P {
5          u p;
6          u pos[8];
7      };
8
9      const u v = sqrt(BN), vv = sqrt(v);
10     vector<bool> isp(v + 1, true);
11     for (u i = 2; i <= vv; ++i)
12         if (isp[i])
13             for (u j = i * i; j <= v; j += i) isp[j] = false;
14
15     const u rsize = BN > 60184 ? BN / (log(BN) - 1.1) :
16         max(1.0, BN / (log(BN) - 1.11)) + 1 + 30;
17     vector<int> primes = {2, 3, 5}; u psize = 3;
18     primes.resize(rsize); vector<P> sprimes;
19
20     u pbeg = 0, prod = 1;
21     for (u p = 7; p <= v; ++p) {
22         if (!isp[p]) continue;
23         if (p <= Q) prod *= p, ++pbeg, primes[psize++] = p;
24         P pp = {p, {}};
25         for (int t = 0; t < 8; ++t) {
26             u j = p <= Q ? p : p * p;
27             while (j % 30 != rs[t]) j += p << 1;
28             pp.pos[t] = j / 30;
29         }
30         sprimes.push_back(pp);
31     }
32
33     vector<u8> pre(prod, 0xFF);
34     for (size_t pi = 0; pi < pbeg; ++pi) {
35         auto &pp = sprimes[pi];
36         const u p = pp.p;
37         for (int t = 0; t < 8; ++t) {
38             const u8 m = ~(1 << t);
39             for (u i = pp.pos[t]; i < prod; i += p) pre[i] &= m;
40         }
41     }
42
43     const u block_size = (L + prod - 1) / prod * prod;
44     vector<u8> block(block_size);

```

```

45     u8* __restrict pblock = block.data();
46     const u M = (BN + 29) / 30;
47
48     for (u beg = 0; beg < M; beg += block_size, pblock -= block_size) {
49         u end = min(M, beg + block_size);
50
51         for (u i = beg; i < end; i += prod)
52             memcpy(pblock + i, pre.data(), min(prod, end - i));
53         if (beg == 0) pblock[0] &= 0xFE;
54
55         for (size_t pi = pbeg; pi < sprimes.size(); ++pi) {
56             auto &pp = sprimes[pi];
57             const u p = pp.p;
58             #pragma GCC unroll 8
59             for (int t = 0; t < 8; ++t) {
60                 u i = pp.pos[t];
61                 const u8 m = ~(1 << t);
62                 for (; i < end; i += p) pblock[i] &= m;
63                 pp.pos[t] = i;
64             }
65         }
66
67         for (u i = beg; i < end; ++i)
68             for (u m = pblock[i]; m > 0; m &= m - 1)
69                 primes[psize++] = i * 30 + rs[__builtin_ctz(m)];
70     }
71
72     while (psize > 0 && primes[psize - 1] > BN) --psize;
73     primes.resize(psize); return primes;
74 }

```

Egcd, Linear Diophantine

```

1  array<ll, 3> eGcd(ll a, ll b) {
2      if (b == 0) return {a, 1, 0};
3      auto [g, x1, y1] = eGcd(b, a % b);
4      return {g, y1, x1 - (a / b) * y1};
5  }
6
7  ax0 + by0 = g ==> ax + by = c
8  x0 *= c/g, y0 *= c/g
9
10 all solutions:
11     x = x0 + k * (b/g),
12     y = y0 - k * (a/g);
13
14 int ceildiv(int a, int b) {
15     if ((a ^ b) >= 0) return (a + b - 1) / b;
16     else return a / b;
17 }
18
19 int flordiv(int a, int b) {
20     if ((a ^ b) >= 0) return a / b;
21     else return (a - b + 1) / b;
22 }
23

```

```

24 non-negative solution:
25     k >= ceildiv(-x*g,b)
26     k <= flordiv(x*g, a)
27
28 bool havenonnegsol(int a, int b, int c, int& x, int& y) {
29     int g = egcd(a, b, x, y);
30     x *= c/g;
31     y *= c/g;
32     int l1 = ceildiv(-x*g, b);
33     int l2 = flordiv(y*g, a);
34     return l1 <= l2;
35 }
36
37 // MOD INV
38 ll modInv(ll a, ll m) {
39     ll x = 1, x1 = 0, q, t, b = m;
40     while(b) {
41         q = a / b;
42         a -= q * b, t = a, a = b, b = t;
43         x -= q * x1, t = x, x = x1, x1 = t;
44     }
45     assert(a == 1);
46     return (x + m) % m;
47 }

```

CRT

```

1  using T = __int128;
2  // ax + by = __gcd(a, b)
3  // returns __gcd(a, b)
4  T extended_euclid(T a, T b, T &x, T &y) {
5      T xx = y = 0;
6      T yy = x = 1;
7      while (b) {
8          T q = a / b;
9          T t = b; b = a % b; a = t;
10         t = xx; xx = x - q * xx; x = t;
11         t = yy; yy = y - q * yy; y = t;
12     }
13     return a;
14 }
15 // finds x such that x % m1 = a1, x % m2 = a2. m1 and m2 may not be coprime
16 // here, x is unique modulo m = lcm(m1, m2). returns (x, m). on failure, m = -1.
17 pair<T, T> CRT(T a1, T m1, T a2, T m2) {
18     T p, q;
19     T g = extended_euclid(m1, m2, p, q);
20     if (a1 % g != a2 % g) return make_pair(0, -1);
21     T m = m1 / g * m2;
22     p = (p % m + m) % m;
23     q = (q % m + m) % m;
24     return make_pair((p * a2 % m * (m1 / g) % m +
25                     q * a1 % m * (m2 / g) % m) % m, m);
26 }

```

MillerRabin

```

1  ll binpower(ll base, ll e, ll mod) {
2      ll result = 1;
3      base %= mod;
4      while (e) {
5          if (e & 1)
6              result = (__uint128_t)result * base % mod;
7              base = (__uint128_t)base * base % mod;
8              e >>= 1;
9      }
10     return result;
11 }
12
13 bool check_composite(ll n, ll a, ll d, int s) {
14     ll x = binpower(a, d, n);
15     if (x == 1 || x == n - 1)
16         return false;
17     for (int r = 1; r < s; r++) {
18         x = (__uint128_t)x * x % n;
19         if (x == n - 1)
20             return false;
21     }
22     return true;
23 };
24
25 bool MillerRabin(long long n) {
26     if (n < 2)
27         return false;
28
29     vector<int> primes = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37};
30     for (auto a : primes)
31         if (n % a == 0)
32             return false;
33
34     int r = 0;
35     long long d = n - 1;
36     while ((d & 1) == 0) {
37         d >>= 1;
38         r++;
39     }
40
41     for (int a : primes) {
42         if (n == a)
43             return true;
44         if (check_composite(n, a, d, r))
45             return false;
46     }
47     return true;
48 }

```

Number of Divisors up to 1e18

```

1  namespace countdivisors {
2      // for one second: 5000 1e18, 100000 1e9, 200000 1e6
3      using ull = unsigned long long;
4      using u128 = __uint128_t;
5      static mt19937_64 rnd(chrono::steady_clock::now().time_since_epoch().count());
6      u128 mul128(ull a, ull b, ull m) { return (u128)a * b % m; }

```

```

7   ull mul_mod(ull a, ull b, ull m) { return (ull)mul128(a, b, m); }
8   ull pow_mod(ull a, ull d, ull m) {
9       ull r = 1;
10      while (d) {
11          if (d & 1) r = mul_mod(r, a, m);
12          a = mul_mod(a, a, m);
13          d >>= 1;
14      }
15      return r;
16  }
17  bool isPrime(ull n) {
18      if (n < 2) return false;
19      for (ull p : vector<ull>({2ULL,3,5,7,11,13,17,19,23,29,31,37}))
20          if (n % p == 0) return n == p;
21      ull d = n - 1, s = 0;
22      while (!(d & 1)) d >>= 1, ++s;
23      for (ull a : vector<ull>({2ULL,325,9375,28178,450775,9780504,1795265022})) {
24          if (a % n == 0) continue;
25          ull x = pow_mod(a, d, n);
26          if (x == 1 || x == n - 1) continue;
27          bool comp = true;
28          for (ull r = 1; r < s; ++r) {
29              x = mul_mod(x, x, n);
30              if (x == n - 1) { comp = false; break; }
31          }
32          if (comp) return false;
33      }
34      return true;
35  }
36  ull pollards_rho(ull n) {
37      if (n % 2 == 0) return 2;
38      while (true) {
39          ull c = uniform_int_distribution<ull>(1, n - 1)(rnd);
40          auto f = [&](ull x){ return (mul_mod(x, x, n) + c) % n; };
41          ull x = rnd() % n, y = x, d = 1;
42          while (d == 1) {
43              x = f(x); y = f(f(y));
44              d = gcd<ull>(x > y ? x - y : y - x, n);
45          }
46          if (d < n) return d;
47      }
48  }
49  void factor(ull n, map<ull,int>& cnt) {
50      if (n < 2) return;
51      if (isPrime(n)) { cnt[n]++; return; }
52      ull d = pollards_rho(n);
53      factor(d, cnt);
54      factor(n/d, cnt);
55  }
56  ull ans(ull n) {
57      map<ull,int> cnt;
58      factor(n, cnt);
59      ull res = 1;
60      for (auto [p, e] : cnt) res *= (e + 1);
61      return res;
62  }

```

```
63 }
```

n mod 1 + n mod 2 + n mod 3 + + n mod m

```
1 // n and m tends to(1e13) => time complexity(sqrt(n))
2 void solve(int idx){
3     int k=n/idx;
4     int st=n%idx%mod;
5     int l=n/(k+1);
6     int len=(idx-l)%mod;
7     ans=(ans + st*len%mod + k%mod*(len*(len-1)/2%mod)%mod)%mod;
8     if(l) solve(l);
9 }
```

n-th Fib Number

```
1 int fast_Fibonacci(int n) {
2     int i = 1, h = 1, j = 0, k = 0, t;
3     while (n > 0) {
4         if (n % 2 == 1)
5             t = j * h, j = i * h + j * k + t, i = i * k + t;
6         t = h * h, h = 2 * k * h + t, k = k * k + t, n = n / 2;
7     }
8     return j;
9 }
```

Long Division

```
1 int a = 23, b = 5, n = 10;
2 vector<int> s;
3 for (int i = 0; i < n; i++) {
4     unsigned long long k = a / b; // Integer division: gets the next digit
5     a -= b * k;                    // Remove the part already represented
6     a *= 10;                       // Shift remainder left (for next digit)
7     s.push_back(k);                // Store the digit
8 }
```

Floor Values

```
1 // code to get all different values of floor(n/i)
2 for (ll l = 1, r = 1; (n/l); l = r + 1) { // O(sqrt)
3     r = (n/(n/l));
4     // q = (n/l), process the range [l, r]
5 }
```

Combinatorics

```
1 namespace combinatorics {
2     vector<mint> fac_(1, 1), inv_(1, 1);
3     void build(int N) {
4         N += 10;
5         fac_.assign(N, 1);
6         inv_.assign(N, 1);
```

```

7         for (int i = 2; i < N; i++)
8             fac_[i] = fac_[i-1] * i;
9         inv_[N-1] = 1/fac_[N-1];
10        for (int i = N-2; i > 1; i--)
11            inv_[i] = inv_[i+1] * (i+1);
12    }
13    inline mint nCr(int n, int r) {
14        if(n < 0 || r < 0 || r > n) return 0;
15        return fac_[n] * inv_[r] * inv_[n - r];
16    }
17
18    inline mint nCr(int64_t n, int64_t r) {
19        if(n < 0 || r < 0 || r > n) return 0;
20        r = min<int64_t>(r, n - r);
21        mint ans = inv_[r];
22        for(int64_t i = n - r + 1; i <= n; i++) ans = ans * i;
23        return ans;
24    }
25    inline mint nPr(int n, int r) {
26        if(n < 0 || r < 0 || r > n) return 0;
27        return fac_[n] * inv_[n - r];
28    }
29    inline mint stars_andBars(int n, int r){
30        return nCr(n + r - 1, r - 1);
31    }
32    inline mint catalan(int n) {
33        if(n < 0) return 0;
34        return fac_[2 * n] * inv_[n] * inv_[n + 1];
35    }
36 }
37 // using namespace combinatorics;
38 auto pre = []() { combinatorics::build(); return 0; }();

```

nCr, nPr without precomputation

```

1  ll nCr(ll n, ll r) {
2      if (r < 0 || r > n) return 0;
3      r = min(r, n-r);
4      ll ret = 1;
5      for (int i = 0; i < r; i++)
6          ret = ret * (n-i) / (i+1);
7      return ret;
8  }
9  ll nPr(int n, int r) {
10     if (r < 0 || r > n) return 0;
11     ll ret = 1;
12     for (int i = 0; i < r; ++i) {
13         ret *= (n - i);
14     }
15     return ret;
16 }

```

NCR table

```

1 void preprocess_nCr() {

```

```

2     for (int n = 0; n < N; n++) {
3         C[n][0] = C[n][n] = 1;
4         for (int r = 1; r < n; r++) {
5             C[n][r] = (C[n - 1][r - 1] + C[n - 1][r]) % MOD;
6         }
7     }
8 }

```

Catalan numbers

```

1  const int MOD = ....
2  const int MAX = ....
3  int catalan[MAX];
4  void init() {
5      catalan[0] = catalan[1] = 1;
6      for (int i=2; i<=n; i++) {
7          catalan[i] = 0;
8          for (int j=0; j < i; j++) {
9              catalan[i] += (catalan[j] * catalan[i-j-1])% MOD;
10             if (catalan[i] >= MOD)
11                 catalan[i] -= MOD;
12         }
13     }
14 }
15
16 // 1- Number of correct bracket sequence consisting of n opening and n
17 // closing brackets.
18
19 // 2- The number of rooted full binary trees with n+1 leaves (vertices
20 // are not numbered).
21 // A rooted binary tree is full if every vertex has
22 // either two children or no children.
23
24 // 3- The number of ways to completely parenthesize n+1 factors.
25
26 // 4- The number of triangulations of a convex polygon with n+2 sides
27 // (i.e. the number of partitions of polygon into disjoint triangles by
28 // using the diagonals).
29
30 // 5- The number of ways to connect the 2n points on a circle to form n
31 // disjoint chords.
32
33 // 6- The number of non-isomorphic full binary trees with n internal
34 // nodes (i.e. nodes having at least one son).
35
36 // 7- The number of monotonic lattice paths from point (0,0) to point
37 // (n,n) in a square lattice of size nxn,
38 // which do not pass above the main diagonal (i.e. connecting (0,0) to
39 // (n,n)).
40
41 // 8- Number of permutations of length n that can be stack sorted
42 // (i.e. it can be shown that the rearrangement is stack sorted if and
43 // only if there is no such index i<j<k, such that ak<ai<aj).
44
45 // 9- The number of non-crossing partitions of a set of n elements.
46
47 // 10- The number of ways to cover the ladder 1..n using n rectangles

```


Matrix Exponentiation

```
1  template <typename T>
2  struct Matrix {
3      int n, m;
4      vector<T> mat;
5
6      Matrix(int n, int m) : n(n), m(m) {
7          mat.assign(n * m, 0);
8      }
9      Matrix(int n) : n(n), m(n) {
10         mat.assign(n * n, 0);
11     }
12
13     void identity() {
14         assert(n == m);
15         fill(mat.begin(), mat.end(), 0);
16         for (int i = 0; i < n; i++) {
17             mat[i * m + i] = 1;
18         }
19     }
20
21     T* operator[](int r) { return &mat[r * m]; }
22     const T* operator[](int r) const { return &mat[r * m]; }
23
24     Matrix operator*(const Matrix& o) const {
25         assert(m == o.n);
26         Matrix res(n, o.m);
27
28         for (int i = 0; i < n; i++) {
29             for (int k = 0; k < m; k++) {
30                 T val = mat[i * m + k]; if (val == 0) continue;
31                 for (int j = 0; j < o.m; j++) {
32                     res[i][j] = (res[i][j] + 1ll * val * o[k][j]) % mod;
33                     // res[i][j] = res[i][j] + val * o[k][j];
34                 }
35             }
36         }
37         return res;
38     }
39
40     Matrix operator+(const Matrix& o) const {
41         assert(o.n == n && o.m == m);
42         Matrix ret(n, m);
43         for (int i = 0; i < n; i++)
44             for (int j = 0; j < m; j++) {
45                 ret[i][j] = ((*this)[i][j] + o[i][j]) % mod;
46                 // ret[i][j] = (*this)[i][j] + o[i][j];
47             }
48         return ret;
49     }
50
51     Matrix pow(ll k) const {
52         assert(n == m);
53         Matrix res(n), base = *this;
54         res.identity();
55         while (k) {
```

```

56         if (k & 1) res = res * base;
57         base = base * base;
58         k >>= 1;
59     }
60     return res;
61 }
62 };

```

K-th permutation

```

1  ll fac[21];
2  void preFac() {
3      fac[0] = 1;
4      for(int i = 1; i <= 20; ++i)
5          fac[i] = fac[i - 1] * i;
6  }
7
8  vector<int> kth_permutation(int n, ll k) { // n^2
9      vector<int> a(n), ans;
10     iota(a.begin(), a.end(), 1);
11     for (int i = n; i >= 1; i--) {
12         ll f = fac[i - 1];
13         ans.push_back(a[k / f]);
14         a.erase(a.begin() + k / f);
15         k %= f;
16     }
17     return ans;
18 }

```

Permutation Index

```

1  ll permutation_index(vector<int>& p) { // n^2
2      int n = int(p.size());
3      vector<int> a(n);
4      iota(a.begin(), a.end(), 1);
5
6      ll k = 0;
7      for (int i = 0; i < n; ++i) {
8          int j = int(find(a.begin(), a.end(), p[i]) - a.begin());
9          k += j * fac[n - 1 - i];
10         a.erase(a.begin() + j);
11     }
12     return k;
13 }

```

Berlekamp Massey

```

1  const ll MOD = 1e9+7;
2  ll add(ll a, ll b) { return (a + b) % MOD; }
3  ll sub(ll a, ll b) { return ((a - b) % MOD + MOD) % MOD; }
4  ll mul(ll a, ll b) { return (a * b) % MOD; }
5  ll power(ll base, ll exp) {
6      ll res = 1;
7      base %= MOD;
8      while (exp > 0) {

```

```

9         if (exp % 2 == 1) res = mul(res, base);
10        base = mul(base, base);
11        exp /= 2;
12    }
13    return res;
14 }
15 ll modInverse(ll n) {
16     return power(n, MOD - 2); // Valid only if MOD is prime
17 }
18 // Finds the shortest linear recurrence transition array for a given sequence S
19 vector<ll> BerlekampMassey(const vector<ll>& S) {
20     int n = S.size(), L = 0, m = 0;
21     vector<ll> C(n), B(n), T;
22     C[0] = B[0] = 1;
23     ll b = 1;
24
25     for (int i = 0; i < n; i++) {
26         ++m;
27         ll d = S[i] % MOD;
28         for (int j = 1; j <= L; j++) d = add(d, mul(C[j], S[i - j]));
29         if (d == 0) continue;
30
31         T = C;
32         ll coef = mul(d, modInverse(b));
33         for (int j = m; j < n; j++) C[j] = sub(C[j], mul(coef, B[j - m]));
34
35         if (2 * L > i) continue;
36         L = i + 1 - L;
37         B = T;
38         b = d;
39         m = 0;
40     }
41     C.resize(L + 1);
42     C.erase(C.begin());
43     for (auto &x : C) x = sub(0, x);
44     return C;
45 }
46 // Multiplies two polynomials modulo the characteristic polynomial (tr)
47 vector<ll> combine(int n, const vector<ll>& a, const vector<ll>& b, const vector<ll>& tr) {
48     vector<ll> res(n * 2 + 1, 0);
49     for (int i = 0; i < n + 1; i++) {
50         for (int j = 0; j < n + 1; j++) {
51             res[i + j] = add(res[i + j], mul(a[i], b[j]));
52         }
53     }
54     for (int i = 2 * n; i > n; --i) {
55         for (int j = 0; j < n; j++) {
56             res[i - 1 - j] = add(res[i - 1 - j], mul(res[i], tr[j]));
57         }
58     }
59     res.resize(n + 1);
60     return res;
61 }
62 // S contains the initial sequence values, 'tr' is the transition array from BM.
63 // K is 0-indexed, S and tr must be same size
64 ll LinearRecurrence(const vector<ll>& S, const vector<ll>& tr, ll k) { //  $O(N^2 \log K)$ 

```

```

65     assert(S.size() == tr.size());
66     int n = tr.size();
67     if (n == 0) return 0;
68     if (k < n) return S[k] % MOD;
69
70     vector<ll> pol(n + 1), e(pol);
71     pol[0] = e[1] = 1;
72
73     for (++k; k; k /= 2) {
74         if (k % 2) pol = combine(n, pol, e, tr);
75         e = combine(n, e, e, tr);
76     }
77
78     ll res = 0;
79     for (int i = 0; i < n; i++)
80         res = add(res, mul(pol[i + 1], S[i]));
81     return res;
82 }

```

5. Strings

Rolling Hash

```
1 using u64 = uint64_t;
2 mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count() ^
3   (uintptr_t)make_unique<char>().get());
4
5 struct hash61 {
6     static const u64 md = (1LL << 61) - 1;
7     inline static u64 step = (md >> 2) + rng() % (md >> 1);
8     inline static vector<u64> pw = {1};
9
10    int n;
11    vector<u64> pref, suff;
12
13    u64 add(u64 a, u64 b) const { return (a += b) >= md ? a - md : a; }
14    u64 sub(u64 a, u64 b) const { return (a += md - b) >= md ? a - md : a; }
15    u64 mul(u64 a, u64 b) const { return (unsigned __int128)a * b % md; }
16
17    template<class T>
18    hash61(const T& s) : n(s.size()), pref(n + 1), suff(n + 1) {
19        while (pw.size() <= n) pw.push_back(mul(pw.back(), step));
20        pref[0] = suff[n] = 1;
21        for (int i = 0; i < n; i++) pref[i + 1] =
22            add(mul(pref[i], step), s[i]);
23        for (int i = n - 1; i >= 0; i--) suff[i] =
24            add(mul(suff[i + 1], step), s[i]);
25    }
26
27    u64 operator()(int l, int r) const { return sub(pref[r + 1],
28        mul(pref[l], pw[r - l + 1])); }
29    u64 rev(int l, int r) const { return sub(suff[l],
30        mul(suff[r + 1], pw[r - l + 1])); }
31 };
32
33 struct hash61 {
34     static const u64 md = (1LL << 61) - 1;
35     inline static u64 step = (md >> 2) + rng() % (md >> 1);
36
37     inline static u64 power(u64 b, u64 e) {
38         u64 r = 1;
39         while (e) {
40             if (e & 1) r = (unsigned __int128)r * b % md;
41             b = (unsigned __int128)b * b % md;
42             e >>= 1;
43         }
44         return r;
45     }
46
47     inline static u64 inv_step = power(step, md - 2);
48     inline static vector<u64> pw = {1}, ipw = {1};
49
50     int n = 0;
51     vector<u64> pref = {1}, suff = {0};
52
53     u64 add(u64 a, u64 b) const { return (a += b) >= md ? a - md : a; }
```

```

54     u64 sub(u64 a, u64 b) const { return (a += md - b) >= md ? a - md : a; }
55     u64 mul(u64 a, u64 b) const { return (unsigned __int128)a * b % md; }
56
57     hash61() {}
58     template<class T> hash61(const T& s) { for (auto c : s) push_back(c); }
59
60     void push_back(u64 c) {
61         while (pw.size() <= n + 1) {
62             pw.push_back(mul(pw.back(), step));
63             ipw.push_back(mul(ipw.back(), inv_step));
64         }
65         pref.push_back(add(mul(pref.back(), step), c));
66         suff.push_back(add(suff.back(), mul(c, pw[n])));
67         n++;
68     }
69
70     void pop_back() {
71         if (!n) return;
72         pref.pop_back();
73         suff.pop_back();
74         n--;
75     }
76
77     u64 operator()(int l, int r) const { return sub(pref[r + 1], mul(pref[l],
78         pw[r - l + 1])); }
79     u64 rev(int l, int r) const { return mul(sub(suff[r + 1], suff[l]), ipw[l]); }
80 };

```

Tree Hash (rooted/unrooted)

```

1  // tree hash
2  using u64 = uint64_t;
3  mt19937_64 rng = []{
4      u64 time_entropy = chrono::steady_clock::now().time_since_epoch().count();
5      u64 memory_entropy = (uintptr_t)make_unique<char>().get();
6      seed_seq ss{time_entropy, memory_entropy};
7      return mt19937_64(ss);
8  }();
9  u64 SEED = rng();
10 u64 mix(u64 x) {
11     x += SEED + 0x9e3779b97f4a7c15;
12     x = (x ^ x >> 30) * 0xbf58476d1ce4e5b9;
13     x = (x ^ x >> 27) * 0x94d049bb133111eb;
14     return x ^ x >> 31;
15 }
16 u64 treehash(int u, int p, vector<vector<int>>& t) {
17     u64 ret = 1;
18     // do ret = ret * P + mix() if the order is important
19     for (int v : t[u]) if (v ^ p)
20         ret += mix(treehash(v, u, t));
21     return ret;
22 }
23 // unrooted: find centroids and treat them as roots
24 u64 utreehash(vector<vector<int>>& t) {
25     int n = t.size();
26     vector<int> sz(n), cents;

```

```

27     function<void(int, int)> dfs1 = [&](int u, int p) {
28         sz[u] = 1;
29         bool can = 1;
30         for (int v : t[u]) if (v ^ p) {
31             dfs1(v, u);
32             if (sz[v] * 2 > n) can = 0;
33             sz[u] += sz[v];
34         }
35         if (n - sz[u] > n/2) can = 0;
36         if (can) cents.push_back(u);
37     };
38     dfs1(0, 0);
39     return min(treeshash(cents.front(), -1, t), treeshash(cents.back(), -1, t));
40 }

```

Z-Algo

```

1  vector<int> z_algo(string s) {
2      vector<int> z(s.size());
3      for(int i = 1, l = 0, r = 0; i < s.size(); i++) {
4          if(i < r) z[i] = min(r - i, z[i - l]);
5          while(i + z[i] < s.size() && s[z[i]] == s[z[i] + i]) z[i]++;
6          if(i + z[i] > r) r = i + z[i], l = i;
7      }
8      return z;
9  }

```

Largest Lexicographical Substring

```

1  string largestLexSubstring(const string &s) {
2      int n = int(s.size());
3      int i = 0, j = 1, k = 0;
4
5      while (j + k < n) {
6          if (s[i + k] == s[j + k]) k++;
7          else if (s[i + k] < s[j + k])
8              /* change it to > if you want lowest */
9              i = max(i + k + 1, j), j = i + 1, k = 0;
10         else j = j + k + 1, k = 0;
11     }
12
13     return s.substr(i);
14 }

```

Manacher

```

1  auto manacher(const string &t) {
2      string s = "%#";
3      s.reserve(t.size() * 2 + 3);
4      for(char c : t) s += c + "#";
5      s += '$';
6      // t = aabaacaabaa -> s = %a#a#b#a#a#c#a#a#b#a#a#$
7
8      vector<int> res(s.size());
9      for(int i = 1, l = 1, r = 1; i < s.size(); i++) {

```

```

10     res[i] = max(0, min(r - i, res[l + r - i]));
11     while(s[i + res[i]] == s[i - res[i]]) res[i]++;
12     if(i + res[i] > r) {
13         l = i - res[i];
14         r = i + res[i];
15     }
16 }
17 for(auto &i : res) i--;
18 return vector(res.begin() + 2, res.end() - 2); // a#a#b#a#a#c#a#a#b#a#a
19 // get max odd len = res[2 * i]; aba -> i = b
20 // get max even len = res[2 * i + 1]; abba -> i = first b
21 }

```

Aho Corasick Algorithm

```

1  namespace corasick {
2      const int N = +++++;
3      const int SIGMA = +++++;
4
5      int nxt[N][SIGMA], fail_link[N], dict_link[N], match_idx[N], nodes;
6
7      void init() {
8          nodes = fail_link[0] = dict_link[0] = 0;
9          memset(nxt[0], 0, sizeof(nxt[0]));
10         match_idx[0] = -1;
11         nodes++;
12     }
13
14     int create_node() {
15         memset(nxt[nodes], 0, sizeof(nxt[nodes]));
16         fail_link[nodes] = dict_link[nodes] = 0;
17         match_idx[nodes] = -1;
18         return nodes++;
19     }
20
21     int insert(const string& pattern, int id) {
22         int cur = 0;
23         for (char c : pattern) {
24             if (!nxt[cur][c]) {
25                 nxt[cur][c] = create_node();
26             }
27             cur = nxt[cur][c];
28         }
29         if (~match_idx[cur]) return match_idx[cur];
30         return match_idx[cur] = id;
31     }
32
33     void build() {
34         queue<int> q;
35
36         for (int c = 0; c < SIGMA; ++c) {
37             int chi = nxt[0][c];
38             if (chi) {
39                 fail_link[chi] = dict_link[chi] = 0;
40                 q.push(chi);
41             }
42         }

```



```

43
44     while (!q.empty()) {
45         int u = q.front();
46         q.pop();
47
48         for (int c = 0; c < SIGMA; ++c) {
49             int chi = nxt[u][c];
50
51             if (chi) {
52                 fail_link[chi] = nxt[fail_link[u]][c];
53
54                 int fail = fail_link[chi];
55                 if (match_idx[fail] != -1)
56                     dict_link[chi] = fail;
57                 else
58                     dict_link[chi] = dict_link[fail];
59                 q.push(chi);
60             } else
61                 nxt[u][c] = nxt[fail_link[u]][c];
62         }
63     }
64 }
65
66 vector<vector<int>> search(const string& text, const vector<int>& lengths) {
67     vector<vector<int>> ret(lengths.size());
68     for (int cur = 0, i = 0; i < text.length(); ++i) {
69         cur = nxt[cur][text[i]];
70
71         for (int u = cur; u; u = dict_link[u]) {
72             int id = match_idx[u];
73             if (id != -1) {
74                 ret[id].push_back(i - lengths[id] + 1);
75             }
76         }
77     }
78     return ret;
79 }
80 } /* init() insert() build() search() */

```

XOR Trie

```

1  template<typename T>
2  class BinaryTrie {
3  private:
4      vector<array<int, 2>> nodes;
5      vector<int> sz;
6      int numberOfBits;
7      T one = 1;
8
9      bool isNodeAvailable(int cur, bool nxt) {
10         return nodes[cur][nxt] != 0 && sz[nodes[cur][nxt]] > 0;
11     }
12
13     int getNode(T x) {
14         int cur = 0;
15         for (int i = numberOfBits - 1; i >= 0; --i) {
16             bool nxt = (x >> i) & 1;

```

```

17         if (nodes[cur][nxt] == 0) {
18             nodes[cur][nxt] = nodes.size();
19             nodes.push_back({0, 0});
20             sz.push_back(0);
21         }
22         cur = nodes[cur][nxt];
23     }
24     return cur;
25 }
26
27 public:
28     BinaryTrie() : nodes(1, {0, 0}), sz(1, 0), numberOfBits(sizeof(T) * 8) {}
29
30     void insert(T x, int cnt = 1) {
31         int cur = 0;
32         sz[cur] += cnt;
33         for (int i = numberOfBits - 1; i >= 0; --i) {
34             bool nxt = (x >> i) & 1;
35             if (nodes[cur][nxt] == 0) {
36                 nodes[cur][nxt] = nodes.size();
37                 nodes.push_back({0, 0});
38                 sz.push_back(0);
39             }
40             cur = nodes[cur][nxt];
41             sz[cur] += cnt;
42         }
43     }
44
45     void erase(T x, int cnt = 1) { insert(x, -cnt); }
46
47     T getMinXor(T x) {
48         if (sz[0] == 0) throw out_of_range("The trie is empty");
49         int cur = 0;
50         T ret = 0;
51         for (int i = numberOfBits - 1; i >= 0; --i) {
52             bool nxt = (x >> i) & 1;
53             if (isNodeAvailable(cur, nxt)) {
54                 cur = nodes[cur][nxt];
55             } else {
56                 ret |= (one << i);
57                 cur = nodes[cur][!nxt];
58             }
59         }
60         return ret;
61     }
62
63     T getMaxXor(T x) {
64         if (sz[0] == 0) throw out_of_range("The trie is empty");
65         int cur = 0;
66         T ret = 0;
67         for (int i = numberOfBits - 1; i >= 0; --i) {
68             bool nxt = !((x >> i) & 1);
69             if (isNodeAvailable(cur, nxt)) {
70                 ret |= (one << i);
71                 cur = nodes[cur][nxt];
72             } else {
73                 cur = nodes[cur][!nxt];

```

```

74     }
75     }
76     return ret;
77 }
78
79 size_t size() { return sz[0]; }
80 bool contains(T x) { return sz[0] && getMinXor(x) == 0; }
81 int count(T x) { return sz[getNode(x)]; }
82 };

```

Suffix Array

```

1 // O(n log(n))
2 struct suffix {
3     int n;
4     vector<int> p, c, lcp;
5     string s;
6
7     explicit suffix(string _s) : n(int(_s.size()) + 1), s(move(_s)), p(n),
8         c(n), lcp(n - 1) {
9         s += char(0);
10        iota(p.begin(), p.end(), 0);
11        sort(p.begin(), p.end(), [&](int i, int j) { return s[i] < s[j]; });
12        for (int i = 1; i < n; i++) c[p[i]] = c[p[i - 1]] + (s[p[i]] != s[p[i - 1]]);
13        vector<int> nc(n), np(n);
14        int k = 1;
15        while (k < n) {
16            vector<int> f(n + 1);
17            for (int i = 0; i < n; i++) p[i] = (p[i] - k + n) % n, f[c[i] + 1]++;
18            for (int i = 1; i <= n; i++) f[i] += f[i - 1];
19            for (int i = 0; i < n; i++) np[f[c[p[i]]]++] = p[i];
20            swap(p, np), nc[p[0]] = 0;
21            for (int i = 1; i < n; i++)
22                nc[p[i]] = nc[p[i - 1]] +
23                    (c[p[i]] != c[p[i - 1]]
24                     || c[(p[i] + k) % n] != c[(p[i - 1] + k) % n]);
25            swap(c, nc), k <= 1;
26        }
27        for (int i = k = 0; i < n - 1; i++) {
28            int j = p[c[i] - 1];
29            for (; s[i + k] == s[j + k]; k++);
30            if (c[i]) lcp[c[i] - 1] = k;
31            k = max(k - 1, 0);
32        }
33        s.pop_back(), c.pop_back(), n--;
34        p.erase(p.begin()), lcp.erase(lcp.begin());
35        for (auto &i : c) i--;
36    }
37    vector<vector<int>> table;
38    void buildLcp() {
39        int LOG = __lg(n) + 1;
40        table.resize(LOG, vector<int>(n));
41        table[0] = lcp;
42        for (int l = 1; l < LOG; l++) {
43            for (int i = 0; i + (1 << (l - 1)) < n; i++) {

```

```

44         table[l][i] = min(table[l - 1][i], table[l - 1][i + (1 << (l - 1))]);
45     }
46 }
47 }
48 int query(int l, int r) { // 0-based
49     if(l == r) return n - 1 - l;
50     l = c[l], r = c[r];
51     if(l > r) swap(l, r);
52     r--;
53     int len = __lg(r - l + 1);
54     return min(table[len][l], table[len][r - (1 << len) + 1]);
55 }
56 auto count(string const &t) { // O(log(n) * size(t))
57     int l = int(lower_bound(p.begin(), p.end(), -1, [&](int i, int j) {
58         return s.substr(i, min<size_t>(t.size(), n - i)) < t;
59     }) - p.begin());
60     int r = int(lower_bound(p.begin() + l, p.end(), -1, [&](int i, int j) {
61         return s.substr(i, min<size_t>(t.size(), n - i)) <= t;
62     }) - p.begin()) - 1;
63     return pair{l, r};
64 }
65 };

```

Suffix Automaton

```

1 namespace sam {
2     const int N = 4000200, SIGMA = 26; // N two times the max length
3     int sa[N][SIGMA], len[N], fail[N], terminal[N], nodes, lst, C[N];
4
5     void init() {
6         nodes = lst = 0;
7         memset(sa[0], -1, sizeof sa[0]);
8         fail[0] = -1;
9         len[0] = terminal[0] = 0;
10    }
11
12    int create_node() {
13        nodes++;
14        memset(sa[nodes], -1, sizeof sa[nodes]);
15        terminal[nodes] = 0;
16        return nodes;
17    }
18
19    void insert(char ch) {
20        int u = create_node();
21        len[u] = len[lst] + 1;
22        int p, q;
23        for (p = lst; ~p; p = fail[p]) {
24            q = sa[p][ch];
25            if (~sa[p][ch]) break;
26            sa[p][ch] = u;
27        }
28
29        if (p == -1) {
30            fail[u] = 0;
31        } else if (len[p] + 1 == len[q]) {

```

```

32         fail[u] = q;
33     } else {
34         int x = create_node();
35         len[x] = len[p] + 1;
36         memcpy(sa[x], sa[q], sizeof sa[0]);
37         for (int v = p; ~v && sa[v][ch] == q; v = fail[v])
38             sa[v][ch] = x;
39         fail[x] = fail[q];
40         fail[q] = fail[u] = x;
41     }
42     lst = u;
43     C[u] = 1;
44 }
45
46 void insert(const string &s) {
47     for (auto ch: s)
48         insert(ch);
49 }
50
51 string walk(const string &s) {
52     int cur = 0;
53     for (auto ch: s) {
54         if (sa[cur][ch] == -1) return "NO"s;
55         cur = sa[cur][ch];
56     }
57     return "YES";
58 }
59
60 void pre_count() {
61     vector<vector<int>> > vlen(N);
62     for (int i = 0; i < N; i++) vlen[len[i]].push_back(i);
63     for (int l = N - 1; ~l; l--)
64         for (auto x: vlen[l])
65             C[fail[x]] += C[x];
66 }
67
68 int count(const string &s) {
69     int cur = 0;
70     for (auto ch: s) {
71         if (!~sa[cur][ch]) return 0;
72         cur = sa[cur][ch];
73     }
74     return C[cur];
75 }
76
77 vector<ll> count_distinct() {
78     vector<ll> dp(nodes + 1), outdeg(nodes + 1);
79     vector<vector<int>> > indeg(nodes + 1);
80     for (int i = 0; i <= nodes; i++)
81         for (int j = 0; j < SIGMA; j++)
82             if (~sa[i][j]) {
83                 outdeg[i]++;
84                 indeg[sa[i][j]].push_back(i);
85             }
86     queue<int> q;
87     for (int i = 0; i <= nodes; i++)
88         if (!outdeg[i])

```

```

89         q.push(i);
90     while (!q.empty()) {
91         int top = q.front();
92         q.pop();
93         dp[top] = 1;
94         for (int i = 0; i < SIGMA; i++)
95             if (~sa[top][i])
96                 dp[top] += dp[sa[top][i]];
97         for (auto x: indeg[top]) {
98             outdeg[x]--;
99             if (!outdeg[x]) q.push(x);
100         }
101     }
102
103     return dp;
104 }
105
106 ll count_distinct2() {
107     ll ret = 0;
108     for (int i = 1; i <= nodes; i++)
109         ret += len[i] - len[fail[i]];
110     return ret;
111 }
112
113 ll sum_distinct() {
114     vector<ll> d(nodes + 1), ans(nodes + 1), outdeg(nodes + 1);
115     vector<vector<int>> indeg(nodes + 1);
116     queue<int> q;
117     for (int i = 0; i <= nodes; i++) {
118         for (int j = 0; j < SIGMA; j++)
119             if (~sa[i][j]) {
120                 outdeg[i]++;
121                 indeg[sa[i][j]].push_back(i);
122             }
123         if (!outdeg[i]) q.push(i);
124     }
125     while (!q.empty()) {
126         int top = q.front();
127         q.pop();
128
129         d[top] = 1;
130         for (int i = 0; i < SIGMA; i++)
131             if (~sa[top][i]) {
132                 d[top] += d[sa[top][i]];
133                 ans[top] += d[sa[top][i]] + ans[sa[top][i]];
134             }
135
136         for (int x: indeg[top])
137             if (--outdeg[x])
138                 q.push(x);
139     }
140     return ans.front();
141 }
142
143 ll sum_distinct2() {
144     ll ret = 0;

```

```

145     for (int i = 1; i <= nodes; i++) {
146         int a = len[i], b = len[fail[i]] + 1;
147         ll num = a-b+1;
148         ret += num * (a+b)/2;
149     }
150     return ret;
151 }
152
153 string get_kth(ll k) {
154     auto d = count_distinct();
155     string ret;
156     int cur = 0;
157     while (k) {
158         for (int i = 0; i < SIGMA; i++) if (~sa[cur][i]) {
159             if (d[sa[cur][i]] >= k) {
160                 k--;
161                 ret.push_back(i);
162                 cur = sa[cur][i];
163                 break;
164             }
165             k -= d[sa[cur][i]];
166         }
167     }
168     return ret;
169 }
170
171 string smallest_shift(int n) {
172     int cur = 0;
173     string ret;
174     while (n--) {
175         for (int i = 0; i < SIGMA; i++) if (~sa[cur][i]) {
176             cur = sa[cur][i];
177             ret.push_back(i);
178             break;
179         }
180     }
181     return ret;
182 }
183
184 vector<int> vis(N);
185 void dfs(int cur, const vector<ll>& d) {
186     if (vis[cur]) return;
187     for (int i = 0; i < SIGMA; i++) if (~sa[cur][i]) {
188         cout << cur << "," << d[cur]-1 << '□' << sa[cur][i] <<
189             "," << d[sa[cur][i]]-1 << '□' << char(i+'a') << '\n';
190         dfs(sa[cur][i], d);
191     }
192 }
193 void prnt() {
194     auto d = count_distinct();
195     dfs(0, d);
196 }
197 } /* init() insert() */

```

Suffix Automaton Subproblems

6. Dynamic Programming

LIS

```
1  int lis_size(const vector<int>& nums) {
2      vector<int> tail;
3      for (auto x : nums) {
4          auto it = lower_bound(tail.begin(), tail.end(), x);
5          if (it == tail.end()) tail.push_back(x);
6          else *it = x;
7      }
8      return tail.size();
9  }
10
11 int lis_with_sequence(const vector<int>& a,
12                      vector<int>& out_seq)
13 {
14     int n = a.size();
15     vector<int> tail;
16     vector<int> tail_idx;
17     vector<int> prev(n, -1);
18     tail.reserve(n);
19     tail_idx.reserve(n);
20
21     for (int i = 0; i < n; ++i) {
22         int x = a[i];
23         auto it = lower_bound(tail.begin(), tail.end(), x);
24         int pos = it - tail.begin();
25         if (it == tail.end()) {
26             tail.push_back(x);
27             tail_idx.push_back(i);
28         }
29         else {
30             *it = x;
31             tail_idx[pos] = i;
32         }
33         if (pos > 0)
34             prev[i] = tail_idx[pos-1];
35     }
36
37     int lis_len = tail.size();
38     out_seq.clear();
39     for (int cur = tail_idx[lis_len - 1]; cur >= 0; cur = prev[cur])
40         out_seq.push_back(a[cur]);
41     reverse(out_seq.begin(), out_seq.end());
42
43     return lis_len;
44 }
```

0/1 Knapsack

```
1  int knapsack(int W, vector<int> &val, vector<int> &wt) {
2      vector<int> dp(W + 1, 0);
3      for (int i = 1; i <= wt.size(); i++)
4          for (int j = W; j >= wt[i - 1]; j--)
5              dp[j] = max(dp[j], dp[j - wt[i - 1]] + val[i - 1]);
```



```

6
7
8     return dp[W];
9 }
10
11 int unbounded_knapSack(int capacity, vector<int> &val, vector<int> &wt) {
12     vector<int> dp(capacity + 1, 0);
13     for (int i = val.size() - 1; i >= 0; i--) {
14         for (int j = 1; j <= capacity; j++) {
15             int take = 0;
16             if (j - wt[i] >= 0) {
17                 take = val[i] + dp[j - wt[i]];
18             }
19             int noTake = dp[j];
20             dp[j] = max(take, noTake);
21         }
22     }
23     return dp[capacity];
24 }

```

Edit Distance

```

1 string a, b; cin >> a >> b;
2 int na = a.size(), nb = b.size();
3 int dp[na+1][nb+1];
4 memset(dp, 0, sizeof dp);
5
6 for (int i = 1; i <= na; i++)
7     dp[i][0] = i;
8 for (int j = 1; j <= nb; j++)
9     dp[0][j] = j;
10 for (int i = 1; i <= na; i++) {
11     for (int j = 1; j <= nb; j++) {
12         dp[i][j] = min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]}) + 1;
13         if (a[i-1] == b[j-1])
14             dp[i][j] = min(dp[i][j], dp[i-1][j-1]);
15     }
16 }
17 cout << dp[na][nb];

```

Longest Common Subsequence

```

1 void lcs(char* X, char* Y, int m, int n)
2 {
3     int L[m + 1][n + 1];
4     for (int i = 0; i <= m; i++) {
5         for (int j = 0; j <= n; j++) {
6             if (i == 0 || j == 0)
7                 L[i][j] = 0;
8             else if (X[i - 1] == Y[j - 1])
9                 L[i][j] = L[i - 1][j - 1] + 1;
10            else
11                L[i][j] = max(L[i - 1][j], L[i][j - 1]);
12        }
13    }
14    int index = L[m][n];

```

```

15
16 char lcs[index + 1];
17 lcs[index] = '\0';
18 int i = m, j = n;
19 while (i > 0 && j > 0) {
20     if (X[i - 1] == Y[j - 1]) {
21         lcs[index - 1]
22         = X[i - 1];
23         i--;
24         j--;
25         index--;
26     }
27     else if (L[i - 1][j] > L[i][j - 1])
28         i--;
29     else
30         j--;
31 }
32 }

```

Shortest Common Supersequence

```

1 while (i > 0 && j > 0) {
2     if (str1[i - 1] == str2[j - 1]) {
3         result.push_back(str1[i - 1]);
4         i--;
5         j--;
6     } else if (dp[i - 1][j] > dp[i][j - 1]) {
7         result.push_back(str1[i - 1]);
8         i--;
9     } else {
10        result.push_back(str2[j - 1]);
11        j--;
12    }
13 }
14 while (i > 0) {
15     result.push_back(str1[i - 1]);
16     i--;
17 }
18 while (j > 0) {
19     result.push_back(str2[j - 1]);
20     j--;
21 }
22 reverse(result.begin(), result.end());
23 return result;

```

Count Subsets Sum to K

```

1 int perfectSum(vector<int> &arr, int target) {
2     int n = arr.size();
3     vector<int> dp(target + 1, 0), ndp(target + 1, 0);
4     dp[0] = 1;
5     for (int i = 1; i <= n; i++) {
6         ndp = dp;
7         for (int j = 0; j <= target; j++) {
8             if (j >= arr[i - 1]) {
9                 ndp[j] += dp[j - arr[i - 1]];

```

```

10         }
11     }
12     dp = ndp;
13 }
14 return ndp[target];
15 }

```

Hamiltonian Paths

```

1 int rec(int u, int vis) {
2     vis |= (1 << u);
3     if (u == n-1)
4         return __builtin_popcount(vis) == n;
5
6     int& ans = dp[vis][u];
7     if (~ans) return ans;
8     // dpid[u][vis] = 2;
9     ans = 0;
10    each(ver, g[u])
11        if (!((vis >> ver) & 1))
12            ans = (ans + rec(ver, vis | (1 << ver)))%mod;
13    return ans;
14 }

```

Kadane

```

1 template<class T>
2 array<ll, 3> kadane(const vector<T>& arr) {
3     ll mx = LLONG_MIN, csum = 0;
4     ll s = 0, e = 0, ts = 0, sz = arr.size();
5     for (ll i = 0; i < sz; i++) {
6         if (csum + arr[i] > arr[i]) {
7             csum += arr[i];
8         } else {
9             csum = arr[i];
10            ts = i;
11        }
12        if (csum > mx) {
13            mx = csum;
14            s = ts;
15            e = i;
16        }
17    }
18    return {mx, s, e};
19 }

```

Dynamic max subarray sum

```

1 struct info {
2     int sum, pref, suff, ans, mnelement = -1e4-10;;
3     info(int x) {
4         sum = pref = suff = x;
5         // ans = max<int>(x, 0);
6         ans = x; // if empty subarray is not allowed
7     }

```

```

8     info() { // default value
9         sum = pref = suff = ans = mnelement;
10    }
11    friend info operator+(const info &l, const info &r) {
12        info ret;
13        ret.sum = l.sum + r.sum;
14        ret.pref = max(l.pref, l.sum + r.pref);
15        ret.suff = max(r.suff, r.sum + l.suff);
16        ret.ans = max({l.ans, r.ans, l.suff + r.pref});
17        return ret;
18    }
19 };

```

Meet in the middle

```

1  vector<int> a;
2  auto get_subset_sums = [&](int l, int r) -> vector<ll> {
3      int len = r - l + 1;
4      vector<ll> res;
5      for (int i = 0; i < (1 << len); i++) {
6          ll sum = 0;
7          for (int j = 0; j < len; j++)
8              if (i & (1 << j))
9                  sum += a[l + j];
10
11          res.push_back(sum);
12      }
13      return res;
14 };
15 vector<ll> left = get_subset_sums(0, n / 2 - 1);
16 vector<ll> right = get_subset_sums(n / 2, n - 1);
17 sort(left.begin(), left.end());
18 sort(right.begin(), right.end());
19 ll ans = 0;
20 for (ll i : left) {
21     auto low_iterator = lower_bound(right.begin(), right.end(), x - i);
22     auto high_iterator = upper_bound(right.begin(), right.end(), x - i);
23     ans += high_iterator - low_iterator;
24 }

```

Digit DP

```

1  using ull = unsigned long long int;
2  #define int ull
3  pair<int, int> dp[2][16];
4  bool vis[2][16];
5  string num;
6  pair<int, int> rec(int idx, bool U) {
7      if (idx == num.size()) return {0, 1};
8      pair<int, int> &ret = dp[U][idx];
9      if (vis[U][idx]) return ret;
10     vis[U][idx] = 1;
11     ret = {0, 0};
12     int D = U ? num[idx] - '0' : 9;
13     for (int d = 0; d <= D; d++) {
14         auto [x, y] = rec(idx+1, U&d==D);

```

```
15         ret.first += x + d * y;  
16         ret.second += y;  
17     }  
18     return ret;  
19 }
```

7. Bit Manipulation

Xor Basis

```
1  template<const int Log = 30, typename T = ll>
2  struct basis {
3      int sz = 0;
4      array<T, Log> a{};
5      void add(T x) {
6          if(sz == Log) return;
7          int i;
8          while(x) {
9              if(!a[i = __lg(x)]) return sz++, void(a[i] = x);
10             x ^= a[i];
11         }
12     }
13     T reduce(T x) {
14         if(sz == Log) return 0;
15         T res = 0;
16         int i;
17         while(x) {
18             if(a[i = __lg(x)]) x ^= a[i];
19             else res |= T(1) << i, x ^= T(1) << i;
20         }
21         return res;
22     }
23     bool find(T x) {
24         if(sz == Log) return true;
25         int i;
26         while(x) {
27             if(a[i = __lg(x)]) x ^= a[i];
28             else return false;
29         }
30         return true;
31     }
32     void clear() {
33         if(sz) a.fill(0), sz = 0;
34     }
35     T getMax(T r = 0) {
36         for(int i = Log - 1; i >= 0; i--) r = max(r ^ a[i], r);
37         return r;
38     }
39
40     T find_k(size_t k, T base_val = 0) {
41         assert(k < 1ULL << sz);
42         T curr = base_val;
43         for(int i = Log - 1, b = sz - 1; i >= 0; i--) {
44             if(a[i]) {
45                 if((k >> b & 1) ^ (curr >> i & 1)) curr ^= a[i];
46                 b--;
47             }
48         }
49         return curr;
50     }
51
52     T getMaxBounded(T limit, T r = 0) {
53         T best = -1;
```

```

54     for (int i = Log - 1; i >= 0; i--) {
55         if (a[i]) {
56             T r1 = r, r2 = r ^ a[i];
57             if ((r1 >> i) & 1) swap(r1, r2);
58
59             if ((limit >> i) & 1) {
60                 T temp = r1;
61                 for (int j = i - 1; j >= 0; j--)
62                     temp = max(temp ^ a[j], temp);
63                 best = max(best, temp);
64                 r = r2;
65             } else
66                 r = r1;
67         } else {
68             int bit_r = (r >> i) & 1;
69             int bit_l = (limit >> i) & 1;
70
71             if (bit_l == 1 && bit_r == 0) {
72                 T temp = r;
73                 for (int j = i - 1; j >= 0; j--)
74                     temp = max(temp ^ a[j], temp);
75                 best = max(best, temp);
76                 r = -1;
77                 break;
78             }
79             if (bit_l == 0 && bit_r == 1) {
80                 r = -1;
81                 break;
82             }
83         }
84     }
85     if (r != -1) best = max(best, r);
86
87     return best;
88 }
89
90 friend basis operator+(basis const &l, basis const &r) {
91     if(l.sz == Log) return l;
92     if(r.sz == Log) return r;
93     auto res = l;
94     for(int i = 0; i < Log; i++) if(r.a[i]) res.add(r.a[i]);
95     return res;
96 }
97 };

```

N-SAT

```

1 struct nsat {
2     int n;
3     vector<vector<int>> lit, g;
4     vector<int> occ, pos, neg, val, stk, lvl;
5
6     nsat(int n) : n(n), g(2 * n), occ(2 * n), val(n, -1) {}
7
8     // Pass a list of pairs: {variable_index, is_true}
9     // Example: { {0, 1}, {1, 0} } -> (x_0 OR NOT x_1)

```

```

10 void add_clause(initializer_list<pair<int, int>> vars) {
11     vector<int> c;
12     for (auto [u, is_true] : vars) {
13         int L = (u << 1) | (!is_true);
14         c.push_back(L);
15         g[L].push_back(lit.size());
16         occ[L]++;
17     }
18     lit.push_back(c);
19 }
20
21 void apply(int L) {
22     val[L >> 1] = !(L & 1);
23     stk.push_back(L);
24     for (int c : g[L]) {
25         if (pos[c]++ == 0)
26             for (int u : lit[c]) occ[u]--;
27     }
28     for (int c : g[L ^ 1]) neg[c]++;
29 }
30
31 void undo() {
32     int L = stk.back();
33     stk.pop_back();
34     val[L >> 1] = -1;
35     for (int c : g[L]) {
36         if (--pos[c] == 0)
37             for (int u : lit[c]) occ[u]++;
38     }
39     for (int c : g[L ^ 1]) neg[c]--;
40 }
41
42 bool deduce(int &q_head) {
43     while (q_head < stk.size()) {
44         int L = stk[q_head++];
45         for (int c : g[L ^ 1]) {
46             if (pos[c] > 0) continue; // Clause already satisfied
47             if (neg[c] == lit[c].size()) return false; // Conflict found
48
49             if (neg[c] + 1 == lit[c].size()) { // Unit clause triggered
50                 for (int u : lit[c]) {
51                     if (val[u >> 1] == -1) {
52                         apply(u);
53                         break;
54                     }
55                 }
56             }
57         }
58     }
59     return true;
60 }
61
62 bool ok() {
63     pos.assign(lit.size(), 0);
64     neg.assign(lit.size(), 0);
65     val.assign(n, -1);

```



```

66     stk.clear();
67     lvl.clear();
68     int q_head = 0;
69
70     // Force evaluate base unit clauses
71     for (int c = 0; c < lit.size(); ++c) {
72         if (lit[c].empty()) return false;
73         if (lit[c].size() == 1) {
74             int L = lit[c][0];
75             if (val[L >> 1] == -1) apply(L);
76             else if (val[L >> 1] != !(L & 1)) return false;
77         }
78     }
79
80     // Sort variables by frequency for fast O(1) branching
81     vector<int> order(n);
82     iota(order.begin(), order.end(), 0);
83     sort(order.begin(), order.end(), [&](int a, int b) {
84         return occ[a << 1] + occ[a << 1 | 1] > occ[b << 1] + occ[b << 1 | 1];
85     });
86
87     while (true) {
88         if (deduce(q_head)) {
89             // Find next unassigned variable to branch on
90             int s = -1;
91             for (int v : order) {
92                 if (val[v] == -1) { s = v; break; }
93             }
94
95             // If all variables are assigned, we found a satisfying assignment
96             if (s == -1) return true;
97
98             lvl.push_back(stk.size());
99             apply(s << 1); // Guess that variable 's' is True
100         } else {
101             // Conflict! Backtrack to last decision
102             if (lvl.empty()) return false;
103
104             int backtrack_idx = lvl.back();
105             lvl.pop_back();
106
107             int last_guess = stk[backtrack_idx];
108             while (stk.size() > backtrack_idx) undo();
109
110             q_head = stk.size();
111             apply(last_guess ^ 1); // Flip the guess
112         }
113     }
114 }
115 };

```

Max Xor Subset In Range

```

1 // Given queries L,R find max XOR subset in range
2 const int N = 5e5 + 5;
3 vector<pair<int,int>>qqq[N];

```

```

4  int ans[N],basis[22],arr[N],last[22];
5
6  void add(int ind,int x){
7      for (int i = 21; i >= 0; --i) {
8          if((x >> i & 1) == 0)continue;
9          if(ind > last[i]){
10             swap(x,basis[i]);
11             swap(ind,last[i]);
12         }
13         x ^= basis[i];
14     }
15 }
16 int query(int ind){
17     int ret = 0;
18     for (int i = 21; i >= 0; --i) {
19         if(last[i] >= ind){
20             ret = max(ret,ret ^ basis[i]);
21         }
22     }
23     return ret;
24 }
25
26 void solve() {
27     int n;cin >> n;
28     memset(last,-1,sizeof last);
29     for (int i = 0; i < n; ++i) {
30         cin >> arr[i];
31     }
32     int q;cin >> q;
33     for (int i = 0; i < q; ++i) {
34         int l,r;cin >> l >> r;
35         l--,r--;
36         qq[rr].emplace_back(l,i);
37     }
38     for (int i = 0; i < n; ++i) {
39         add(i,arr[i]);
40         for(auto &x:qq[i]){
41             int l = x.fi;
42             int ind = x.se;
43             ans[ind] = query(l);
44         }
45     }
46     for (int i = 0; i < q; ++i) {
47         cout << ans[i] << endl;
48     }
49 }

```

and (&) in range [l, r]

```

1  ll andRange(ll l, ll r) {
2      ll ans=0, msb = -1;
3      while(r>0) { r >>= 1; msb++;}
4      for(ll i= msb; ~i; i--) {
5          ll la = 1;
6          if((l&(la << i)) == (r&(la << i)))
7              ans += (l&(la << i));

```

```

8         else
9             break;
10    }
11    return ans;
12 }

```

Bit Twiddle Permute

```

1 int bit_twiddle_permute(int v) { // next integer that has _pop_count(v) bits
2     int t = v | (v - 1);
3     int w = (t + 1) | (((~t & -~t) - 1) >> (__builtin_ctz(v) + 1));
4     return w;
5 }

```

8. Game Theory and Sequences

Mex Calculator

```
1 class mex_calculator {
2     map<int, int> count;
3     set<int> missing;
4     public:
5     mex_calculator(const vector<int>& arr, int upper_bound) { // n log n
6         for (int x : arr)
7             count[x]++;
8         for (int i = 0; i <= upper_bound + 1; ++i)
9             if (count[i] == 0)
10                 missing.insert(i);
11     }
12     void insert(int x) { // log
13         count[x]++;
14         if (count[x] == 1)
15             missing.erase(x);
16     }
17     void remove(int x) { // log
18         if (count[x] == 0) return;
19         count[x]--;
20         if (count[x] == 0)
21             missing.insert(x);
22     }
23     int get_mex() { // 1
24         return *missing.begin();
25     }
26 };
```

Remove Game

```
1 void solve() {
2     int n; cin >> n;
3     vector<int> v(n);
4     getv(v);
5     ll diff[n+1][n+1]{}; // mx diff p1-p2 on interval [l, r]
6     for (int l = n-1; ~l; l--) {
7         for (int r = 0; r < n; r++) {
8             // dp[l][r] = mx( v[l]-dp[l+1][r], v[r]-dp[l][r-1] )
9             if (l == r)
10                 diff[l][r] = v[l];
11             else diff[l][r] = max<ll>(v[l] - diff[l+1][r], v[r]-diff[l][r-1]);
12         }
13     }
14
15     cout << (accumulate(all(v), 0ll) + diff[0][n-1])/2;
16 }
```

K-th Balanced Bracket Sequence

```
1 //O(n^2)
2 string kth_balanced(int n, int k) {
3     vector<vector<int>> d(2*n+1, vector<int>(n+1, 0));
```

```

4      d[0][0] = 1;
5      for (int i = 1; i <= 2*n; i++) {
6          d[i][0] = d[i-1][1];
7          for (int j = 1; j < n; j++)
8              d[i][j] = d[i-1][j-1] + d[i-1][j+1];
9          d[i][n] = d[i-1][n-1];
10     }
11     string ans;
12     int depth = 0;
13     for (int i = 0; i < 2*n; i++) {
14         if (depth + 1 <= n && d[2*n-i-1][depth+1] >= k)
15             {
16                 ans += '(';
17                 depth++;
18             } else {
19                 ans += ')';
20                 if (depth + 1 <= n)
21                     k -= d[2*n-i-1][depth+1];
22                 depth--;
23             }
24     }
25     return ans;
26 }

```

Next Balanced Sequence

```

1  bool next_balanced_sequence(string & s) { // O(n)
2      int n = s.size();
3      int depth = 0;
4      for (int i = n - 1; i >= 0; i--) {
5          if (s[i] == '(')
6              depth--;
7          else
8              depth++;
9          if (s[i] == '(' && depth > 0) {
10             depth--;
11             int open = (n - i - 1 - depth) / 2;
12             int close = n - i - 1 - open;
13             string next = s.substr(0, i) + ')' +
14                 string(open, '(') + string(close, ')');
15             s.swap(next);
16             return true;
17         }
18     }
19     return false;
20 }

```

9. Geometry

Geometry Notes

```
1  Generate 2 points on a line
2  // ax + by + c = 0
3  if(a == 0){
4      p1 = {0, -1.0 * c/b};
5      p2 = {1, -1.0 * c/b};
6  }
7  else{
8      p1 = {-1.0 * c/a, 0};
9      p2 = {-1.0 * (c + b)/a, 1};
10 }'
```

11

12

13 You're given 4 integers which are the

14 coefficients A B and C of the normal equation of the

15 straight line and a distance value R.

```
16
17 void solve() {
18     double a,b,c,r; cin >> a >> b >> c >> r;
19     double base = sqrt(a * a + b * b);
20     a /= base;
21     b /= base;
22     c /= base;
23     cout << setprecision(15) << a << ' ' << b << ' ' << c + r << endl;
24     cout << setprecision(15) << a << ' ' << b << ' ' << c - r << endl;
25 }
```

26

27 Area of the sector without an angle = $(l * r) / 2$

28 The length of the arc $l = (\text{theta} / 360) * 2 * \pi * r$

29

30 The area of the parallelogram = the cross-product of

31 2 adjacent sides = 2 * area of the triangle made by 3 points.

32

33 Given 1 side of the Pythagorean Triangle ... Get the missing 2 sides :

```
34 ll n;
35 cin >> n;
36 if (n == 1 || n==2)
37     cout << -1;
38 else if (n & 1)
39     cout << (n * n + 1) / 2 << " " << (n * n - 1) / 2;
40 else
41     cout << n * n / 4 + 1 << " " << n * n / 4 - 1;
42
```

43 In triangle abc angle bac $\cos(\text{theta}) = (b^2 + c^2 - a^2) / (2 * b * c)$

44

45 n = number of sides of a regular polygon

46 S = side length of the polygon

47 ap = apothem the distance from the center of the polygon to the middle of any side

48 r = radius of the polygon which is the distance from the center of the polygon to any corner.

49 p = perimeter of the polygon

50

```
51 p = S * n
52 ap = S / (2 * tan(180/n)) = r * cos(180/n)
53 r = S / (2 * sin(180/n)) = ap / cos(180/n)
54 Area = (p * ap)/2 ,  $(S^2 * n) / (4 * \tan(180/n)) = ap^2 * n * \tan(180/n)$ 
```

```

55     = (r^2 * n * sin(360/n))/2
56
57 sin(2*theta) = 2 * sin(theta) * cos(theta)
58 cos(2*theta) = cos(theta)^2 - sin(theta)^2 = 2 * cos(theta)^2 - 1
59     = 1 - 2 * sin(2*theta)^2
60 sin(theta)^2 = (1 - cos(2 * theta))/2
61 cos(theta)^2 = (1 + cos(2 * theta))/2
62 tan(2*theta) = (2 * tan(theta)) / (1 - tan(theta)^2)
63
64 Circle intersection r1,r2,d where r1 >= r2
65 If d = r1 + r2 they touch from outside
66 If d = r1 - r2 they touch from inside
67 If r1 - r2 < d < r1 + r2 they intersect in two points
68
69 Plane equation ax + by + cz + d = 0
70 AB = (Bx-Ax,By-Ay,Bz-Az)
71 AC = (Cx-Ax,Cy-Ay,Cz-Az)
72 AB x AC = (a,b,c)
73 a = (By-Ay)*(Cz-Az)-(Cy-Ay)*(Bz-Az)
74 b = (Bz-Az)*(Cx-Ax)-(Cz-Az)*(Bx-Ax)
75 c = (Bx-Ax)*(Cy-Ay)-(Cx-Ax)*(By-Ay)
76 d = -(a*Ax+b*Ay+c*Az)
77
78 // Checks if four points lie in the same plane or not
79 bool samePlane(point a,point b,point c){
80     // a * (b x c) = volume = 0
81     return (a.dot(b.cross(c)) == 0);
82 }
83
84 void solve() {
85     vector<point>v(4);
86     for (int i = 0; i < 4; ++i) {
87         cin >> v[i].x >> v[i].y >> v[i].z;
88     }
89     for (int i = 0; i < 4; ++i) {
90         v[i].x -= v[3].x;
91         v[i].y -= v[3].y;
92         v[i].z -= v[3].z;
93     }
94     cout << (samePlane(v[0],v[1],v[2])) ? "YES":"NO" << endl;
95 }

```

Geometry (X, Y, dot, cross)

```

1  /*
2   conj(a) -> a.imag() *= -1
3   abs(point) distance between (0,0) to this point
4   norm(point) squared magnitude -> real^2 + imag^2
5   hypot(x, y) -> sqrt(x^2 + y^2)
6   arg(vector) angle between this vector and x-axis
7   clamp(a, l, r) == min(r, max(l, a))
8   polar(rho, theta) -> make vector with length rho and angle theta
9   internal angle = (n - 2) * 180 / n
10  number of diagonals n * (n - 3) / 2
11  Area(p) = internal_points_cnt + (boundary_points/2) - 1
12  boundary_point in vector = gcd(|x2-x1|, |y2-y1|) + 1

```

```

13 line have infinity point, segment have to end points
14 vector(x, y) perpendicular to vector(-y, x) and (y, -x)
15 */
16
17 using ll = int64_t;
18
19 using ld = double;
20 using pt = complex<ld>;
21
22 const ll INF = 7e18;
23 const ld EPS = 1e-9;
24 const ld PI = acos(-1);
25
26 #define X real()
27 #define Y imag()
28
29 #define dot(a, b) (conj(a) * (b)).X
30 #define cross(a, b) (conj(a) * (b)).Y
31
32 int sign(ld x) {
33     return (x > EPS) - (x < -EPS);
34 }
35
36 struct compX{
37     bool operator()(pt a, pt b) const {
38         return a.X != b.X ? a.X < b.X : a.Y < b.Y;
39     }
40 };
41 struct compY{
42     bool operator()(pt a, pt b) const {
43         return a.Y != b.Y ? a.Y < b.Y : a.X < b.X;
44     }
45 };
46
47 // ===== line, segment =====
48
49 // projection of pt p onto line ab
50 pt project(pt a, pt b, pt p) {
51     pt ab = b - a;
52     return a + ab * dot(p - a, ab) / norm(ab);
53 }
54
55 // works for any orientation
56 bool onSegment(pt a, pt b, pt p) {
57     return sign(cross(b - a, p - a)) == 0 &&
58         sign(dot(p - a, p - b)) <= 0;
59 }
60
61 // ccw: >0 left, <0 right, =0 collinear
62 int ccw(pt a, pt b, pt c) {
63     return sign(cross(b - a, c - a));
64 }
65
66 // works for any pts
67 ld distanceToLine(pt a, pt b, pt p) {
68     return fabs(cross(b - a, p - a)) / abs(b - a);
69 }

```



```

70 // works for any line
71 ld distanceToLine(ld A, ld B, ld C, pt p) {
72     return fabs(A*p.X + B*p.Y + C) / abs(pt(A, B));
73 }
74
75 // works for any pts
76 ld distanceToSegment(pt a, pt b, pt p) {
77     if (dot(b - a, p - a) < 0) return abs(p - a);
78     if (dot(a - b, p - b) < 0) return abs(p - b);
79     return distanceToLine(a, b, p);
80 }
81
82 // works for intersecting lines (not parallel)
83 pt lineIntersect(pt a, pt b, pt c, pt d) {
84     pt ab = b - a, cd = d - c;
85     return a + ab * (cross(c - a, cd) / cross(ab, cd));
86 }
87
88 // works for all segments (returns intersection pt if exists)
89 bool segmentsIntersect(pt a, pt b, pt c, pt d, pt &inter) {
90     int d1 = ccw(a, b, c), d2 = ccw(a, b, d);
91     int d3 = ccw(c, d, a), d4 = ccw(c, d, b);
92
93     if(d1 * d2 < 0 && d3 * d4 < 0)
94         return inter = lineIntersect(a, b, c, d), true;
95
96     if(d1 == 0 && onSegment(a, b, c)) return inter = c, true;
97     if(d2 == 0 && onSegment(a, b, d)) return inter = d, true;
98     if(d3 == 0 && onSegment(c, d, a)) return inter = a, true;
99     if(d4 == 0 && onSegment(c, d, b)) return inter = b, true;
100
101     return false;
102 }
103
104 // works for any triangle
105 ld triangleArea(pt a, pt b, pt c) {
106     return 0.5 * fabs(cross(b - a, c - a));
107 }
108
109 bool ptInTriangle(pt a, pt b, pt c, pt p) {
110     ld s1 = cross(b - a, p - a);
111     ld s2 = cross(c - b, p - b);
112     ld s3 = cross(a - c, p - c);
113     return (sign(s1) >= 0 && sign(s2) >= 0 && sign(s3) >= 0) ||
114           (sign(s1) <= 0 && sign(s2) <= 0 && sign(s3) <= 0);
115 }
116
117 // angle abc in radians
118 ld angle_abc(pt a, pt b, pt c) {
119     return acos(clamp<ld>(dot(a - b, c - b) / (abs(a - b) * abs(c - b)), -1, 1));
120 }
121
122 // ===== Circles =====
123
124 pair<ld, pt> findCircle(pt a, pt b, pt c) {

```

```

126     pt m1 = (a + b) / 2.0, m2 = (b + c) / 2.0;
127     pt ab = b - a, bc = c - b;
128     pt center = lineIntersect(m1, m1 + pt(-ab.Y, ab.X),
129                               m2, m2 + pt(-bc.Y, bc.X));
130     return {abs(center - a), center};
131 }
132
133 vector<pt> lineCircleIntersect(pt a, pt b, pt center, ld r) {
134     pt ab = b - a, ao = center - a;
135     pt proj = a + ab * dot(ao, ab) / norm(ab);
136     ld d = abs(proj - center);
137     if (d > r + EPS) return {};
138     if (abs(d - r) < EPS) return {proj};
139     ld h = (ld)sqrtl(r*r - d*d);
140     pt dir = ab / abs(ab);
141     return {proj + dir * h, proj - dir * h};
142 }
143
144 // in 0, 1, 2 pts
145 vector<pt> circleCircleIntersect(pt c1, ld r1, pt c2, ld r2) {
146     ld d = abs(c2 - c1);
147     if(d > r1 + r2 + EPS || d < abs(r1 - r2) - EPS) return {};
148     if(abs(d) < EPS && abs(r1 - r2) < EPS) return vector(3, c1); // infinity intersection
149
150     ld a = (r1*r1 - r2*r2 + d*d) / (2 * d), h2 = r1*r1 - a*a;
151     if (h2 < -EPS) return {};
152
153     pt dir = (c2 - c1) / d, p = c1 + dir * a;
154     if (abs(h2) < EPS) return {p};
155     ld h = sqrt(h2);
156     pt offset = dir * pt(0, 1) * h;
157     return {p + offset, p - offset};
158 }
159
160 pair<ld, pt> minimumEnclosingCircle(vector<pt> p) {
161     using circle = pair<ld, pt>;
162     shuffle(p.begin(), p.end(), mt19937(random_device{}()));
163     auto contains = [](circle c, const vector<pt>& pts) {
164         return all_of(pts.begin(), pts.end(),
165                       [&](auto p) {return abs(p - c.second) <= c.first + EPS;});
166     };
167     auto circleFrom2 = [](pt a, pt b) {
168         pt c = (a + b) / 2.0;
169         return circle{abs(a - c), c};
170     };
171     auto circleFrom3 = [](pt a, pt b, pt c) {
172         pt ab = (a + b) / 2.0, ac = (a + c) / 2.0;
173         pt ab_perp = (b - a) * pt(0, 1), ac_perp = (c - a) * pt(0, 1);
174         pt o = lineIntersect(ab, ab + ab_perp, ac, ac + ac_perp);
175         return circle{abs(o - a), o};
176     };
177     vector<pt> R;
178     function<circle(int)> welzl = [&](int n) -> circle {
179         if (n == 0 || R.size() == 3) {
180             if (R.empty()) return {};
181             if (R.size() == 1) return {0, R[0]};

```

```

182         if (R.size() == 2) return circleFrom2(R[0], R[1]);
183         return circleFrom3(R[0], R[1], R[2]);
184     }
185     pt q = p[n - 1];
186     circle D = welzl(n - 1);
187     if (contains(D, {q})) return D;
188     R.push_back(q);
189     auto res = welzl(n - 1);
190     R.pop_back();
191     return res;
192 };
193 return welzl((int)p.size());
194 }
195
196 // ===== polygon =====
197
198 // works for any polygon (returns +1 for ccw, -1 for cw)
199 ld polygonSign(vector<pt>& p) {
200     ld area = 0;
201     int n = (int)p.size();
202     p.push_back(p[0]);
203     for(int i = 0; i < n; ++i) area += cross(p[i], p[i + 1]);
204     p.pop_back();
205     return sign(0.5 * area);
206 }
207
208 // works for any polygon (removes dups, enforces ccw order)
209 void normPolygon(vector<pt>& p) {
210     vector<pt> res;
211     for(auto i : p) if(res.empty() || abs(i - res.back()) > EPS)
212         res.push_back(i);
213
214     if(res.size() > 1 && abs(res.front() - res.back()) < EPS)
215         res.pop_back();
216
217     if(polygonSign(res) < 0) reverse(res.begin(), res.end());
218
219     p = res;
220 }
221
222 // works for simple polygons with integer coordinates
223 ll internalPointsCount(vector<pt>& p) {
224     ll A2 = 0, B = 0;
225     int n = (int)p.size();
226     p.push_back(p[0]);
227     for (int i = 0; i < n; ++i) {
228         pt a = p[i], b = p[i + 1];
229         A2 += ll(a.X * b.Y - a.Y * b.X);
230         B += __gcd((ll)abs(b.X - a.X), (ll)abs(b.Y - a.Y));
231     }
232     p.pop_back();
233     return (abs(A2) - B + 2) / 2;
234 }
235
236 // works for any polygon (cw or ccw, convex or not)
237 ld polygonArea(const vector<pt>& p) {

```

```

238     int n = (int)p.size();
239     ld area = 0;
240     for (int i = 0; i+1 < n; ++i)
241         area += cross(p[i], p[i + 1]);
242     area += cross(p.back(), p.front());
243     return fabsl(area) / 2.0;
244 }
245
246 // works for any polygon (cw or ccw, convex or not)
247 bool ptInPolygon(const vector<pt> &p, pt o) {
248     int in = 0, n = (int)p.size();
249     for (int i = 0; i+1 < n; ++i) {
250         pt a = p[i], b = p[i + 1];
251         if (onSegment(a, b, o)) return true;
252         if (a.Y > o.Y != b.Y > o.Y) {
253             ld x = a.X + (b.X - a.X) *
254                 (o.Y - a.Y) / (b.Y - a.Y);
255             if(x > o.X) in ^= 1;
256         }
257     }
258     {
259         pt a = p.back(), b = p.front();
260         if (onSegment(a, b, o)) return true;
261         if ((a.Y > o.Y) != (b.Y > o.Y)) {
262             ld x = a.X + (b.X - a.X) *
263                 (o.Y - a.Y) / (b.Y - a.Y);
264             if(x > o.X) in ^= 1;
265         }
266     }
267     return in;
268 }
269
270
271 // work for simple convex polygon
272 bool ptInConvex(vector<pt> &poly, pt p) {
273     int n = int(poly.size());
274     if(n == 1) return sign(abs(poly[0] - p)) == 0;
275     if(n == 2) return onSegment(poly[0], poly[1], p);
276
277     pt f = poly[0];
278
279     if(sign(cross(poly[1] - f, p - f)) < 0 ||
280        sign(cross(poly[n - 1] - f, p - f)) > 0) return false;
281
282     int l = 1, r = n - 1;
283     while(r > l + 1) {
284         int mid = (l + r) >> 1;
285         if(sign(cross(poly[mid] - f, p - f)) > 0) l = mid;
286         else r = mid;
287     }
288     return ptInTriangle(f, poly[l], poly[r], p);
289 }
290
291 // works for any simple polygon (cw or ccw)
292 pt polygonCentroid(const vector<pt>& p) {
293     ld A = 0, c;

```

```

294     pt C(0, 0);
295     int n = (int)p.size();
296     pt cur, nxt;
297     for (int i = 0; i+1 < n; ++i) {
298         cur = p[i], nxt = p[i + 1];
299         c = cross(cur, nxt);
300         A += c;
301         C += (cur + nxt) * c;
302     }
303     cur = p.back(), nxt = p.front();
304     c = cross(cur, nxt);
305     A += c;
306     C += (cur+nxt) * c;
307
308     A *= 0.5;
309     if (abs(A) < EPS) return C;
310     return C / (6.0 * A);
311 }

```